

第 13 章: while and for Loops (while 与 for 循环)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章为全书"过程式语句 (procedural statements)"巡礼画上句号, 正式讲解 Python 的两大循环语句 while 与 for, 以及配套的 break、continue、pass、循环 else 和 range、zip、enumerate 三个循环辅助内置函数。它是下一章"迭代协议 (iteration protocol) 与列表推导 (list comprehensions)"的直接铺垫——for 循环之所以能遍历一切可迭代对象, 底层秘密将在下一章揭晓。

13.1 开篇引言

英文原文

This chapter concludes our tour of Python procedural statements by presenting the language's two main looping constructs—statements that repeat an action over and over:- while / else: The most general looping statement, which can handle repetitive tasks of all kinds- for / else: A specialized loop designed for stepping through the items in any "iterable" object easily We

've met and used both of these informally already, but we'll fill in additional usage details here. While we're at it, we'll also study a few less prominent statements used within loops, such as `break` and `continue`, the `loop else`, and cover some built-ins commonly used with loops, such as `range`, `zip`, and `enumerate`. Although the `while` and `for` statements covered here are the primary syntax provided for coding repeated actions, there are additional looping operations and concepts in Python. Because of that, the iteration story is continued in the next chapter, where we'll explore the related ideas of Python's iteration protocol (used by the `for` loop) and list comprehensions (a close cousin to the `for` loop). Later chapters explore even more exotic iteration tools such as generators and functional tools like `map`, `filter`, and `reduce`. For now, though, let's keep things simple.

中文翻译

本章通过介绍 Python 的两大循环 (looping) 构造——一遍又一遍重复某个动作的语句——来为我们的 Python 过程式语句之旅画上句号：- `while` / `else`：最通用的循环语句，可以处理各种重复性任务- `for` / `else`：一种专用循环，专为轻松遍历任何“可迭代 (iterable)”对象中的项而设计我们已经非正式地见过并使用过这两者，但这里将补充更多的使用细节。顺便我们还会研究几个只在循环内部有意义的次要语句，如 `break` 和 `continue`、循环 `else`，并介绍一些常与循环配合使用的内置函数，如 `range`、`zip` 和 `enumer`

ate。尽管这里介绍的 while 和 for 语句是为重复动作编码的主要语法，Python 中还有其他循环操作和概念。正因如此，迭代的故事将在下一章继续：在那里我们将探讨 Python 迭代协议（iteration protocol）（供 for 循环使用）以及列表推导（list comprehensions）（for 循环的近亲）等相关概念。后面的章节还会探索更奇特的迭代工具，如生成器（generators）以及 map、filter、reduce 等函数式工具。不过现在，先让我们保持简单。

深度理解

- 核心概念：循环（loop）是三大程序控制流（顺序、分支、循环）的最后一块拼图。本章结束时，你将能用 Python 写出任何"重复性"逻辑。
- 底层视角：注意作者把 for 称作"generic iterator（通用迭代器）"——它的本质不是"计数"，而是"逐个取项"。这个视角是理解 Python 迭代模型的关键。
- 全书脉络：本章"keep things simple"（先保持简单），是因为完整的迭代故事要分几章讲完：第 13 章打基础，第 14 章讲迭代协议与推导式，第 20 章讲生成器与 map/filter/reduce 等函数式工具。
- 初学者误区：很多人把 for 当成 C 语言的计数器循环来用（for (int i=0; ...)），这是 ex-C 程序员的"本能"——但 Python 中 for 默认遍历的是项（items）而不是索引（indexes）。本章后半部分会反复强调这一点。
- 生活例子：while 像"等水烧开"——只要没开就一直等，条件自己掌控；for 像"翻看一本书"——从第一页翻到最

后一页，每页都过一遍，不需要自己数页码。

13.2 while 循环 (while Loops)

13.2.1 通用格式 (General Format)

英文原文

Python's while statement is the most general iteration construct in the language. In simple terms, it repeatedly executes an associated block of statements as long as a test at the top keeps evaluating to a true value. It is called a "loop" because control keeps looping back to the start of the statement until the test becomes false. When the test does become false, control passes to the statement that follows the while block. The net effect is that the loop's body is executed repeatedly while the test at the top is true. If the test is false to begin with, the body never runs and the while statement is skipped. In its most complex form, the while statement consists of a header line with a test expression, a body of one or more normally indented statements, and an optional else part that is executed if control exits the loop without a break statement being encountered. Python keeps evaluating the test at the top and executing the statements nested in the loop body until the test returns a false value:

中文翻译

Python 的 while 语句是这门语言中最通用的迭代构造。简单来说，只要顶部的测试持续求值为真值，它就反复执行与之关联的一个语句块。它被称为"循环 (loop)"，是因为控制流不断绕回语句的开头，直到测试变为假。当测试确实变为假时，控制流传递给 while 块之后的语句。净效果是：只要顶部测试为真，循环体就被反复执行。如果测试一开始就是假，循环体一次都不会运行，整个 while 语句被跳过。在最复杂的形式下，while 语句由三部分组成：带测试表达式的头行 (header line)、一个或多个按常规缩进的语句构成的循环体，以及一个可选的 else 部分——它只在控制流没有遇到 break 语句的情况下退出循环时执行。Python 会不断计算顶部的测试、执行嵌套在循环体中的语句，直到测试返回假值：

代码分析

```
while test:                #
    statements              #
else:                      # else
    statements              # break
```

- 解释器如何处理：CPython 会把 while 编译成字节码：测试指令 POPJUMPIFFALSE（测试为假则跳到循环外的下一条指令）+ 一组循环体指令 + 循环体末尾无条件回跳的 JUMPABSOLUTE（跳回测试指令）。整个循环在底层就是一个"测试—执行—回跳"的机器级循环。
- else 的定位：else 与 while 关键字垂直对齐（同一缩进级别），而不是与循环体对齐——这是它属于"循环"而非"

循环体"的语法信号。它只在"没有通过 break 退出"这一种情形下运行，13.4 节会专门讲。

·设计思想：while 是"最通用的循环"，因为它只依赖一个布尔条件，循环体里可以写任何语句。通用 = 灵活 = 需要你自己负责所有细节（计数器、索引、退出条件）。与之相对，for 把这些细节自动化了。

13.2.2 示例 (Examples)

英文原文

To illustrate, let's look at a few simple while loops in action. The first, which consists of a print statement nested in a while loop, just prints a message forever. Recall that True is just a custom version of the integer 1 and always stands for a Boolean true value; because the test is always true, Python keeps executing the body forever or until you stop its execution. This sort of behavior is usually called an infinite loop—it's not really immortal, but you may need a Ctrl+C key combination to forcibly terminate it: `python >>> while True: ... print('Type Ctrl+C to stop me!')` The next example keeps slicing off the first character of a string until the string is empty and hence false (and begins omitting the REPL's ... prompts for easier copy and paste where possible). It's typical to test an object directly like this instead of using the more verbose equivalent (`while x != ''`), though later in this chapter you'll see other ways to step through the items in a string more e

asily with a for loop: python >>> x = 'code'>>> while x: # 当 x 非空时... print(x, end='') # 打印下一个字符... x = x[1:] # 切掉 x 的第一个字符... code ode de e Note the end="" keyword argument used here to place all outputs on the same line separated by a space; see Chapter 11 if you've forgotten why this works as it does. This will probably leave your REPL's input prompt at the end of the output's line; type Enter (or your keyboard's or app's equivalent) to reset if desired. The following code counts from the value of a up to, but not including, b. Loops like this are often used to generate object indexes. You'll also see an easier way to do this with a Python for loop and the built-in range function later: python >>> a = 0; b = 10 >>> while a < b: # 编码计数器循环的一种方式... print(a, end='') ... a += 1 # 或者 a = a + 1 ... 0 1 2 3 4 5 6 7 8 9 Finally, notice that Python doesn't have what some languages call a "do until" loop statement. However, we can simulate one with a test and break at the bottom of the loop body, so that the loop's body is always run at least once: python while True: ...loop body... # 循环体至少运行一次if test: break # 在底部测试循环退出 To fully understand how this structure works, we need to move on to the next section's coverage of break.

中文翻译

为了说明问题，我们来看几个运行中的简单 while 循环。第一个循环由一个嵌套在 while 循环中的 print 语句组成，

只是永远打印一条消息。回忆一下：True 只是整数 1 的一个定制版本，始终代表布尔真值；因为测试永远为真，Python 会一直执行循环体，直到你终止它。这种行为通常被称为无限循环（infinite loop）——它并非真的“不朽”，只是你可能需要按下 Ctrl+C 组合键来强制终止它：python

```
>>> while True: ... print('Type Ctrl+C to stop me!')
```

下一个示例不断切片（slicing）掉字符串的第一个字符，直到字符串为空（因而为假）为止（这里开始省略 REPL 的 ... 提示符，方便复制粘贴）。像这样直接测试对象本身是很典型的写法，而不写更冗长的等价形式（while x != ''）；不过本章稍后你会看到用 for 循环遍历字符串项的更简单方式：

```
python >>> x = 'code'>>> while x: # 当 x 非空时... print(x, end='') # 打印下一个字符... x = x[1:] # 切掉 x 的第一个字符... code ode de e
```

注意这里使用了 end="" 关键字参数，让所有输出放在同一行并用空格分隔；如果你忘了它为什么这样工作，请复习第 11 章。这大概会让 REPL 的输入提示符停留在输出行末尾；如果需要，按回车（或你的键盘/应用的等价键）即可重置。下面的代码从 a 的值开始计数，一直数到 b（不含 b）。这类循环常用于生成对象索引。稍后你还会看到用 Python for 循环和内置 range 函数实现同样目标的更简单方式：

```
python >>> a = 0; b = 10>>> while a < b: # 编码计数器循环的一种方式... print(a, end='') ... a += 1 # 或者 a = a + 1 ... 0 1 2 3 4 5 6 7 8 9
```

最后，请注意 Python 没有某些语言所谓的“do until”（先执行后判断）循环语句。不过，我们可以用循环体底部的测试加 break 来模拟一个，使循环体总是至少运行一次：

```
python while True: ...loop body... # 循环体至少运行一次if test: break # 在底部测试循环退出要完全理解这个结
```


构的原理，我们需要进入下一节，学习 break 的内容。

深度理解

·核心概念：while 的三个典型用途——无限循环（等待用户/外部信号）、按条件收缩（切片字符串直到空）、计数器循环（生成索引）。

·底层实现：字符串是对象，`x[1:]` 每次生成一个新字符串（切片是新对象），旧对象失去引用后被垃圾回收（garbage collection, GC）回收。`'code' -> 'ode' -> 'de' -> 'e' -> ''`，空字符串的布尔值为假，循环自然结束。

·为什么这样设计：Python 的真值测试（truth testing）看的是“对象自身的布尔值”——空字符串、空列表、0、None 都是假。所以 `while x:` 比 `while x != ''` 更简洁，也更符合 Python 的风格。

·实际问题：`while True:` 无限循环 + 底部 break 模拟 “do-until”，保证循环体至少执行一次——这是“先读取再判断”场景（如读文件、用户输入）的标准写法。

·初学者误区：① 忘记更新循环变量导致死循环（比如忘了 `x = x[1:]` 或 `a += 1`）；② 以为 `a += 1` 是“改了一个数字变量”，其实它是“把 a 绑定到新整数对象”——整数是不可变（immutable）对象。

·生活例子：无限循环像“深夜电台”——不关掉就一直播；条件循环像“剥洋葱”——一层一层剥，剥到没有为止。

13.3 break、continue、pass 与循环 else (break, continue, pass, and the Loop else)

13.3.1 总览与循环通用格式 (Overview and General Loop Format)

英文原文

Now that we've seen a few Python loops in action, it's time to take a look at two simple statements that have a purpose only when nested inside loops—the break and continue statements. While we're looking at oddballs, we will also study the loop else clause here because it is intertwined with break, as well as Python's empty placeholder statement pass, which is not tied to loops but falls into the category of simple one-word statements. In Python:- break Jumps out of the closest enclosing loop (past the entire loop statement)- continue Jumps to the top of the closest enclosing loop (to the loop's header line)- pass Does nothing at all: it's an empty statement placeholder- Loop else block Runs if and only if the loop is exited normally (i.e., without hitting a break) Factoring in break and continue statements, the general format of the while loop looks like this: python while test: statements if test: break # 立即退出循环, 如果有 else 则跳过if test: continue # 现在跳到循环顶部的测试else: statements # 如果没有遇到'break' 则在退出时运行 break and continue sta

tements can appear anywhere inside the while (or per ahead, for) loop's body, but they are usually coded further nested in an if test to take action in response to some condition. Let's turn to a few simple examples to see how these statements come together in practice.

中文翻译

既然我们已经看过了几个实际运行的 Python 循环，现在是时候看看两个只在嵌套于循环内部时才有意义的简单语句了——break 和 continue 语句。既然在观察这些“怪家伙”，我们也会在这里学习循环 else 子句，因为它与 break 纠缠在一起；还有 Python 的空占位语句 pass，它与循环无关，但属于简单的单词语句（one-word statements）范畴。在 Python 中：

- break：跳出最近的封闭循环（越过整个循环语句）
- continue：跳到最近的封闭循环的顶部（循环的头行）
- pass：什么都不做：它是一个空的语句占位符（placeholder）
- 循环 else 块：当且仅当循环正常退出时运行（即没有遇到 break）

把 break 和 continue 语句考虑进来后，while 循环的通用格式如下：

```
python while test:
    statements
    if test: break # 立即退出循环，如果有 else 则跳过
    if test: continue # 现在跳到循环顶部的测试
else:
    statements # 如果没有遇到'break' 则在退出时运行
```

break 和 continue 语句可以出现在 while（或如后所述，for）循环体内的任何位置，但它们通常被进一步嵌套在 if 测试中，以响应某种条件采取行动。下面我们看几个简单示例，看看这些语句在实践中如何配合使用。

代码分析

```
while test:
    statements
    if test: break          # else
    if test: continue      #
else:
    statements              # 'break'
```

·解释器如何处理：break 对应字节码无条件跳转到循环块之后（同时标记"跳过了 else 块"）；continue 对应字节码跳回循环头部的测试指令。跳转目标地址在编译期就确定了，所以 break/continue 只能作用于"最近的封闭循环"，这是编译器静态决定的，不是运行时行为。

·为什么放在 if 里：裸写的 break/continue 会让循环只执行一次（break）或永远循环（continue），没有意义。它们的价值在于"满足某条件时改变控制流"——所以 99% 的用法是 if 条件: break/continue。

·关键区别一句话：break 是"出门走人"（离开整栋楼），continue 是"回到队尾重新排队"（只跳过本轮剩余代码）。

深度理解

·核心概念：四个循环控制词：break（跳出循环）、continue（跳到循环顶）、pass（空操作占位）、else（正常退出才执行）。前两个只在循环内有意义。

·底层实现：break 编译为跳转到循环块之后的字节码地址；continue 编译为跳转到循环头部的测试/取下一项指令。二者都是无条件跳转，因此"跳过 else"或"跳过本轮剩

余代码"是编译器的既定行为，而非运行时判断。

·为什么这样设计：C 语言用 goto 实现这些控制，但 goto 无约束、易写出"意大利面条式代码"。Python 用受限的 break/continue 提供"结构化跳转"——只允许跳出循环 (break) 和跳到循环头 (continue)，杜绝任意跳转带来的维护噩梦。

·实际问题：搜索类算法（找到即停）、跳过不符合条件的项、空函数/空类占位、循环后的"未找到"处理——这几件事几乎覆盖了所有循环控制需求。

·初学者误区：① 以为 break 会结束整个程序——其实它只跳出最近的循环；② 在嵌套循环里用 continue，误以为会跳过外层循环——它只作用于最近的循环；③ 把 pass 和 continue 混为一谈——pass 什么都不做（本轮照常继续），continue 跳过本轮剩余代码。

·生活例子：break 是"相亲遇到合适的就收工"，continue 是"这盘菜不辣就换下一盘"，pass 是"电话占线时什么都不干，等着"，else 是"逛完整个商场都没买到，才去网购"。

13.3.2 pass 语句 (pass)

英文原文

Simple things first: the pass statement is a no-operation placeholder that is coded when the syntax requires a statement, but you have nothing useful to say. It is often used to code an empty body for a compound

statement. For instance, if you want to code an infinite loop that does nothing each time through, do it with a pass: `python while True: pass` # 输入 Ctrl+C 来停止我! Because the body is just an empty statement, Python gets stuck in this loop. `pass` is roughly to statements as `None` is to objects—an explicit nothing. Notice that here the while loop's body is on the same line as the header, after the colon; as with if statements, this works only if the body isn't a compound statement (and doesn't contain one). This example does nothing forever. It probably isn't the most useful Python program ever written (unless you want to warm up your laptop or phone on a cold winter's day), but it's tough to come up with a better pass example at this point in the book; it's not a commonly used tool. You'll see other places where pass makes a bit more sense later—for instance, to ignore exceptions caught by try statements and to define empty class objects with attributes that behave like "structs" and "records" in other languages. Though partly a preview, a pass is also sometime coded to mean "to be filled in later," to stub out the bodies of functions temporarily: `python def func1(): pass` # 以后在这里添加真正的代码 `def func2(): pass` We can't leave the body empty without getting a syntax error, so we say pass instead.

中文翻译

先从简单的说起：pass 语句是一个无操作（no-operation）占位符，当语法要求必须有一个语句、但你无话可说时使用它。它常用于为复合语句编写空的循环体。例如，如果你想编写一个每轮什么都不做的无限循环，用 pass 即可：python while True: pass # 输入 Ctrl+C 来停止我！因为循环体只是一个空语句，Python 会卡在这个循环里。pass 之于语句，大致就像 None 之于对象——一个显式的“无”。注意这里 while 循环体与头行写在同一行、位于冒号之后；与 if 语句一样，这只有当循环体不是复合语句（且不包含复合语句）时才有效。这个示例会永远什么都不做。它大概是 Python 史上最没用的程序之一（除非你想在寒冷的冬日用笔记本或手机暖手），但在本书目前的进度下，很难想出更好的 pass 示例了；它并不是一个常用的工具。稍后你会看到 pass 更有意义的其他场合——例如忽略 try 语句捕获的异常，以及定义空的 class 对象，让它的属性表现得像其他语言中的“结构体（structs）”和“记录（records）”。虽然有点超前，但 pass 有时也被写成“以后填上”的意思，临时充当函数体的桩（stub）：python def func1(): pass # 以后在这里添加真正的代码 def func2(): pass 我们不能让函数体留空（否则会报语法错误），所以用 pass 顶替。

代码分析

```
while True: pass                                # Ctrl+C
```

·解释器如何处理：while True: pass 编译后是“恒真测试 + 空语句”的无限循环，CPU 会满负荷空转。按下 Ctrl+C 会触发 KeyboardInterrupt 异常，解释器捕获后中断循环。

- 语法细节：冒号后同一行的语句必须是非复合语句——因为复合语句（if/for/while/try 等）内部还需要缩进块，无法压缩到一行。这是语法解析器（parser）的硬性规定

-

```
def func1():  
    pass                                #  
  
def func2():  
    pass
```

- 解释器如何处理：函数体必须至少包含一条语句，否则解析器报语法错误。pass 占位后，函数定义可以正常编译；调用时函数执行"空体"，返回 None。

- 为什么这样设计：Python 用缩进而非花括号表示代码块，"空块"在语法上无法表达——C 语言里 {} 可以是空块，Python 必须显式写 pass。这体现了 Python 的哲学：语法必须清晰、无歧义。

深度理解

- 核心概念：pass 是"显式的无"——满足语法要求但不做任何事。它与 None（对象层面的"无"）形成呼应：一个是语句占位，一个是对象占位。

- 底层实现：pass 不产生任何有效运算的字节码指令，几乎零开销。

- 为什么这样设计：Python 是"块即缩进"的语言，空块无法表示，所以引入 pass。这也催生了"哑占位"文化：写函数/类骨架时先用 pass 占位，之后填充。

·实际问题：① 函数/类的临时空壳 (stub) ； ② except 子句里"有意忽略异常" (先 pass 再回来处理) ； ③ 数据结构"空壳类"，属性即结构体字段。

·初学者误区：① 以为 pass 能"退出循环"——那是 break 的活；② 写了 pass 就不管了，代码"静默失败"——有意的空操作也要留注释说明意图。

·生活例子：pass 像作文里的"此处略去一万字"——占个位置，保证文章结构完整，内容以后补。

13.3.3 省略号字面量替代方案 (The ellipsis-literal alternative)

英文原文

Despite the limited roles, there's a similar, if more obscure, way to achieve the same effect as pass. Python allows an ellipsis, coded as ... (literally, three consecutive dots, not the Unicode character), to appear anywhere an expression can. Because ellipses do nothing by themselves, they can serve as an alternative to the pass statement, especially for code to be filled in later—a sort of Python TBD: `python def func1(): ... # pass` 的替代 `func1() # 如果调用则什么都不做` This works because any expression can appear as a statement (as we learned in Chapter 11), and the ... literal qualifies as an expression. Ellipses can also appear on a statement header by itself, and may be used to initialize variable names if no specific type is required—which also m

akes it an alternative to None: python def func1(): ...
同一行也可以>>> tbd = ... # pass 和 None 的替代>>>
> tbd # Ellipsis 是真实存在的东西（虽然有点怪！）Ellipsis
is This goes well beyond the original intent of ... in slicing
extensions (which, like the @ operator and type hinting, is unused by Python itself), so time will tell if
it rises to challenge pass and None in these inane and
vacuous roles.

中文翻译

尽管用途有限，但还有一种相似（如果说更冷门）的方式能达到与 pass 相同的效果。Python 允许省略号（ellipsis）——写作 ...（字面意义上的三个连续点号，不是 Unicode 字符）——出现在任何表达式可以出现的地方。由于省略号本身什么都不做，它们可以作为 pass 语句的替代品，尤其是用于“以后填充”的代码——一种 Python 式的“待定（TBD）”：python def func1(): ... # pass 的替代func1() #
如果调用则什么都不做这之所以可行，是因为任何表达式都可以作为语句出现（我们在第 11 章学过），而 ... 字面量本身就是一个表达式。省略号也可以单独出现在语句头行上，还可以在沒有特定类型要求时用来初始化变量名——这使它也能成为 None 的替代品：python def func1(): ... #
同一行也可以>>> tbd = ... # pass 和 None 的替代>>>
tbd # Ellipsis 是真实存在的东西（虽然有点怪！）Ellipsis
这已经远远超出了 ... 在切片扩展（slicing extensions）中的原始意图（切片扩展与 @ 运算符、类型注解一样，Python 自身并不使用），所以时间会告诉我们，它是否能在这些无聊空洞的角色里向 pass 和 None 发起挑战。

代码分析

```
def func1():  
    ...                # pass  
func1()                #
```

·解释器如何处理：... 是一个真实的字面量，它被求值为内置单例对象 Ellipsis (Ellipsis 类型的唯一实例)。单独一行 ... 就是"表达式语句"——求值后丢弃结果，效果等同 pass。在 REPL 里输入 tbd 会打印出 Ellipsis，证明它确实是个对象。

·底层机制：... 是 Ellipsis 单例的语法糖，就像 True/False/None 是各自单例的语法糖一样。它的主要"正经"用途是 NumPy 等库的多维切片 (arr[..., 0])，但在占位场景下它只是个哑巴表达式。

深度理解

·核心概念：... 是 Ellipsis 单例的字面量，可以作为 pass/None 的替代品，尤其流行于"待实现"占位（很多现代库源码里 def f(): ... 就是这么写的）。

·底层实现：Ellipsis 是 types.EllipsisType 的唯一实例，全局只有一个，任何地方的 ... 都是同一个对象——这也是"单例模式"的经典案例。

·为什么这样设计：Python 允许任何表达式作为语句，所以 ... 天然能当占位符用。它比 pass 多了一个特性：有值 (Ellipsis)，因此还能初始化变量。

·实际问题：函数/类的"待办桩"、抽象方法占位（配合作者的教学示例：def func1(): ... 比 pass 更醒目地传达"此

处留白")。NumPy 多维数组切片 `arr[..., None]` 是它唯一的"本职"用途。

·初学者误区：① 把 ... 当成"注释"——它是表达式，会被求值；② 在旧代码里见到 `pass` 就以为过时了——... 只是风格选择，二者语义等价。

·生活例子：... 像简历上的"—"或问卷上的"N/A"——明示"此处留白，稍后填写"，比什么都不写更有信息量。

13.3.4 continue 语句 (continue)

英文原文

The `continue` statement causes an immediate jump to the top of a loop. It's often used to avoid statement nesting, as in the next example that uses `continue` to skip odd numbers. This code prints all even numbers less than 10 and greater than or equal to 0. Recall that 0 means false and `%` is the remainder-of-division (modulus) operator, so this loop counts down to 0, skipping numbers that aren't multiples of 2—and prints 8 6 4 2 0: `python x = 10 while x: x -= 1 # 或者 x = x - 1 if x % 2 != 0: continue # 奇数? 跳过打印 print(x, end = ' ') Because continue jumps to the top of the loop, you don't need to nest the print statement here inside an if test; the print is only reached if the continue is not run.`

中文翻译

`continue` 语句会导致立即跳转到循环的顶部。它常用于避免语句嵌套，就像下一个用 `continue` 跳过奇数的示例。这段代码打印所有小于 10 且大于等于 0 的偶数。回忆一下：0 表示假，`%` 是取余（模）运算符，所以这个循环倒数到 0，跳过所有不是 2 的倍数的数字——并打印 8 6 4 2 0：python x = 10 while x: x -= 1 # 或者 x = x - 1 if x % 2 != 0: continue # 奇数？跳过打印 print(x, end='') 因为 `continue` 会跳到循环顶部，所以这里不需要把 `print` 语句嵌套在 `if` 测试里；只有当 `continue` 没有执行时，`print` 才会被走到。

代码分析

```
x = 10
while x:
    x -= 1                # x = x - 1
    if x % 2 != 0: continue #
    print(x, end=' ')
```

·解释器如何处理：循环从 `x=10` 开始，每次先减 1，然后测试奇偶：`x=9` 时 `9 % 2 == 1`（非零即真）→ 执行 `continue`，跳过 `print`，直接回到 `while x` 的测试；`x=8` 时 `8 % 2 == 0`（0 为假）→ 跳过 `continue`，执行 `print(8)`。如此往复直到 `x=0`，`while 0` 测试为假，循环结束。输出为 8 6 4 2 0。

·反直觉点：`if x % 2 != 0` 为真意味着“是奇数”，`continue` 把奇数挡在 `print` 之外——过滤逻辑是“反向写”的。这正是 `continue` 的价值：让“继续处理的代码”保持平铺，

不层层缩进。

·设计思想：continue 消除了"守卫型 if 嵌套"。没有 continue 时你得写 `if x % 2 == 0: print(x, end='')`——多一层缩进。循环体逻辑越复杂，continue 的"防嵌套"收益越大。

深度理解

·核心概念：continue = "立即开始下一轮迭代"，本轮剩余代码全部跳过。

·底层实现：continue 编译为跳到循环头测试指令的无条件跳转。在 while 中它跳回测试语句，在 for 中它跳回"取下一项"的指令（这正是 for/while 中 continue 语义差异的根源）。

·为什么这样设计：作者把它比作受限的"go to"。Python 没有 goto 关键字，但 continue 本质就是一次受控跳转——所以对可读性的警告同样适用：滥用会让循环难以跟踪。

·实际问题：数据处理流水线中最常用的"过滤"手法：`for item in data: if 不合法(item): continue`，让合法路径平铺直叙。

·初学者误区：① 在循环末尾写 continue——完全多余（本轮本来就要结束）；② 忘了 while 循环里 continue 会跳过循环体底部对循环变量的更新，导致死循环（经典错误：`i=0; while i<10: if ...: continue; i+=1`）。

·生活例子：质检流水线——传送带上不符合标准的零件直接"不下线继续走"，只有合格的才被机械臂抓走加工。

13.3.5 嵌套代码的替代方案 (The nested-code alternative)

英文原文

If all this sounds similar to a "go to" in other languages, it should. Python has no "go to" statement, but because `continue` lets you jump about in a program, many of the warnings about readability and maintainability you may have heard about "go to" apply. `continue` should probably be used sparingly, especially when you're first getting started with coding. For instance, the last example might be clearer if the `print` were nested under the `if`: `python x = 10 while x: x -= 1 if x % 2 == 0: # 偶数? 打印print(x, end='')` Later in this book, you'll also learn that `raise` and `try/except` can also emulate "go to" statements in limited and structured ways; stay tuned for more on this technique in Chapter 36 where you will learn how to use it to break out of multiple nested loops, a feat not possible with the next section's topic alone.

中文翻译

如果这一切听起来很像其他语言中的"go to (跳转)", 那感觉没错。Python 没有"go to"语句, 但因为 `continue` 让你能在程序中跳来跳去, 很多你听过的关于"go to"的可读性和可维护性警告同样适用。`continue` 应该谨慎使用, 尤其是刚开始学编程的时候。例如, 上一个示例如果把 `print`

嵌套在 if 下面可能会更清晰：python x = 10 while x: x -= 1 if x % 2 == 0: # 偶数? 打印print(x, end='') 在本书后面你还会学到：抛出并捕获异常也能以受限且有结构的方式模拟"go to"；敬请期待第 36 章的技术——你将学到如何用它跳出多个嵌套循环，这是仅靠下一节的主题（break）无法做到的壮举。

代码分析

```
x = 10
while x:
    x -= 1
    if x % 2 == 0:          #
        print(x, end=' ')
```

·与 continue 版的对比：逻辑完全等价，输出都是 8 6 4 2 0。区别只在"过滤思想"：continue 版是"坏的就跳过"，if 嵌套版是"好的才执行"。前者平铺但跳转多，后者嵌套但线性。

·作者的态度：没有绝对对错——简单的过滤用 if 嵌套更易读；循环体内还有大段后续逻辑时，continue 的"提前脱身"能避免深嵌套。原则是"稀疏使用、以可读性为准"。

·前瞻：break 只能跳出一层循环；要同时跳出多重嵌套循环，需要用异常（第 36 章）或提取函数加 return——这是 Python 社区的标准技巧。

深度理解

·核心概念：continue 是"结构化 goto"，能力强大但要克制使用。作者给初学者的建议：先学会用 if 嵌套，再考虑用 continue。

- 底层实现：异常模拟跳转的机制是：raise 异常对象，异常处理器在栈上逐层查找匹配的 except，找到后直接跳转——CPython 里由异常表（exception table）驱动，与 break 的编译期跳转完全不同。
 - 为什么这样设计：Python 的哲学是"显式优于隐式"——跳转要可见、要受限。continue 跳转方向固定，比 goto 安全得多，但仍建议节制。
 - 实际问题：多嵌套循环的"一键退出"难题；回调式代码中的提前返回（用 return 代替 goto 是现代语言的共同方案）。
 - 初学者误区：把 continue 当成"性能优化工具"到处乱用——其实它只是控制流表达方式，性能差异微乎其微，可读性才是重点。
 - 生活例子：goto 像"在公司里随便串门"（没有门禁），continue 像"只允许走指定通道"（结构化跳转），异常像"打投诉电话一键直达上级"（跨层级跳转）。
-

13.3.6 break 语句 (break)

英文原文

The break statement causes an immediate exit from a loop—technically, from the closest enclosing loop, when loops are nested. Because the code that follows it in the loop is not executed if the break is reached, it can sometimes avoid nesting much like continue. For example, here is a simple interactive loop (a takeoff o

n code we studied in Chapter 10) that inputs data with input and exits when the user enters a "stop" line:

```
python >> num = 1 >> while True: ... tool = input(f'{num}) What\'s your favorite language?') ... if tool == 'stop': break ... print('Bravo!' if tool == 'Python' else 'Try again...') ... num += 1 ... 1) What's your favorite language? Java Try again... 2) What's your favorite language? Python Bravo! 3) What's your favorite language? stop
```

Because the break in this terminates the while immediately, there's no reason to nest code below it in an else.

中文翻译

break 语句会导致立即退出循环——严格来说，在循环嵌套时，是从最近的封闭循环中退出。因为一旦执行到 break，循环中它后面的代码就不会执行，所以它有时能像 continue 一样避免嵌套。例如，这里有一个简单的交互式循环（脱胎于第 10 章研究过的代码），用 input 输入数据，当用户输入一行 "stop" 时退出：python >>> num = 1 >>> while True: ... tool = input(f'{num}) What\'s your favorite language?') ... if tool == 'stop': break ... print('Bravo!' if tool == 'Python' else 'Try again...') ... num += 1 ... 1) What's your favorite language? Java Try again... 2) What's your favorite language? Python Bravo! 3) What's your favorite language? stop 因为这里的 break 会立即终止 while 循环，所以没有理由把它下面的代码嵌套在 else 中。

代码分析

```
>>> num = 1
>>> while True:
...     tool = input(f'{num}) What\'s your favorite language? ')
...     if tool == 'stop': break
...     print('Bravo!' if tool == 'Python' else 'Try again...')
...     num += 1
... 
```

·解释器如何处理：这是标准的"无限循环 + 中途 break"交互模式。每轮：① 调 input() 阻塞等待用户输入（底层走系统调用读取 stdin）；② 若输入'stop' 则 break 跳出——后续的 print 和 num += 1 全部跳过；③ 否则打印评判（条件表达式'Bravo!' if ... else ... 一步完成），编号 num 加一。

·为什么不需要 else：break 之后不会再有什么"正常退出"的路径（测试恒真），循环只可能经 break 退出——所以下方的代码直接平铺在循环外即可，无需任何 else 结构。

·字符串细节：What\'s 里的反斜杠转义 \' 是为了让撇号出现在单引号字符串里；而 f-string 的 {num} 会在每轮动态填入当前编号——注意 num 的初始值 1 在首轮 input 之前就被使用，所以第一行提示是 1)。

深度理解

·核心概念：break = "立即终止循环"，控制流直接落到整个循环语句之后。它与 continue 的差别：一个离开循环，一个只离开本轮。

- 底层实现：break 编译为跳转到循环块结束地址的指令（循环的 except/跳出点），且会置"跳过 else"标志。在 while True 结构中，它是唯一合法的退出路径。
 - 为什么这样设计：交互式输入、菜单循环、事件循环（GUI 框架主循环）都要"运行中动态决定何时停止"——while True + break 是表达"先跑起来再说"最自然的方式。
 - 实际问题：用户输入直到"quit"、读取数据直到 EOF、轮询直到条件满足——这些"直到.....为止"的循环全部用这个模式。
 - 初学者误区：① break 放在 if 外面——条件恒真，循环一次就结束，代码白写；② 以为 break 能跳出多个嵌套循环——它只跳一层（想跳多层要用异常或标志变量，第 36 章）。
 - 生活例子：break 像跑步比赛里的"退出赛道"——一旦决定，立刻离开，后面还有几圈都不管了。
-

13.3.7 命名赋值替代方案 (The named-assignment alternative)

英文原文

That said, it's also possible to use the newer `:=` expression we met in "Named Assignment Expressions" to crunch this example—albeit, at the expense of its role as a break demo. While you should judge for yourself, the net effect is concise but may at least flirt with unreadability: `python >>> num = 1 >>> while (tool :`

= input(f'{num}) What\'s your favorite language?')) != 'stop': ... print('Bravo!' if tool == 'Python' else 'Try again...') ... num += 1 ... 1) What's your favorite language? Python Bravo! Nesting := within := as in the following , however, could easily incite pitchforks and torches (in fact, the full one-liner here is too wide for this book!). Unless you can defend this in a court of your code-reuse peers, just say no: python num = 0 while (tool := input(f'{(num := num + 1)}) What\'s your favorite language?')) != 'stop': Preview: in Chapter 36, you'll see that input also raises an exception at end-of-file (e.g., if the user enters Ctrl+Z on Windows or Ctrl+D on Unix); wrapping input in try statements allows users to respond this way too.

中文翻译

话虽如此，也可以使用我们在“命名赋值表达式（Named Assignment Expressions）”一节中见过的较新的 := 表达式来压缩这个示例——虽然，代价是牺牲了它作为 break 示例的角色。你应该自行判断，净效果是简洁，但至少是在跟“不可读”调情：python >>> num = 1 >>> while (tool := input(f'{num}) What\'s your favorite language?')) != 'stop': ... print('Bravo!' if tool == 'Python' else 'Try again...') ... num += 1 ... 1) What's your favorite language? Python Bravo! 不过，像下面这样把 := 嵌套在 := 里，很容易招来“草叉和火把”（事实上，这里的一行式代码太宽，本书都放不下！）。除非你能在代码复用同伴的“法庭”上为它辩护，否则还是拒绝吧：python num = 0 while (tool :

= input(f'{{(num := num + 1)}} What\'s your favorite language?')) != 'stop': 预告：在第 36 章你会看到，input 在文件结尾处还会抛出异常（例如用户按 Windows 的 Ctrl + Z 或 Unix 的 Ctrl + D）；把 input 包在 try 语句中，就能让用户用这种方式应答。

代码分析

```
>>> num = 1
>>> while (tool := input(f'{num}')) What\'s your favorite language? ...
...     print('Bravo!' if tool == 'Python' else 'Try again...')
...     num += 1
```

·解释器如何处理：海象运算符（walrus）:= 在表达式中完成“赋值 + 求值”两步：先把 input(...) 的结果赋给 tool，再拿这个值去和 'stop' 比较。while 的测试表达式每轮都会执行一次这个赋值，所以 tool 在循环体里可直接使用——少写一行 tool = input(...)。

·作者的态度：代码“少了”，但信息密度高了——读者必须先意识到“哦，测试表达式里有赋值副作用”。这是可读性与简洁性的经典权衡，作者明确倾向后者谨慎使用。

·嵌套海象的灾难：{(num := num + 1)} 在 f-string 内部先自增再格式化——一行代码塞进三件事（自增、取输入、比较），可读性趋近于零，作者称之为“招来草叉和火把”，坚决不推荐。

深度理解

·核心概念：:= 海象运算符（Python 3.8+ 引入）允许在表达式内赋值，常用于 while 测试和 if 条件，避免重复调

用函数。

·底层实现：`:=` 在编译期是一个独立的赋值指令，赋值的目标进入当前作用域；在 `f-string` 内使用时，`f-string` 表达式会编译成普通表达式——所以嵌套海象“能跑”，但语义晦涩。

·为什么这样设计：PEP 572 的动机是消除“先赋值后比较”的重复代码（如 `data = f(); if data:` 里 `f()` 被调用或变量被写两次的问题），但 PEP 作者也反复警告：不要为了炫技而嵌套。

·实际问题：读取文件直到空、读取输入直到特殊值、正则匹配直到无结果——一切“循环里既要结果又要继续”的场景。

·初学者误区：① 分不清 `=`（赋值语句）与 `:=`（赋值表达式）——`=` 不能出现在表达式里，`:=` 不能单独做语句；

② 在列表推导里滥用海象导致变量意外泄漏。

·生活例子：`:=` 像“买菜时顺便称重”——买之前先把重量记录好再决定买不买；嵌套海象像“买完菜还在同一句话里算完账、找完零”——能行，但结账员看着都晕。

13.3.8 循环 `else` (Loop `else`)

英文原文

When combined with the `loop else` clause, the `break` statement can often eliminate the need for the search status flags used in other languages. In abstract terms the `break` in the following skips the `else` on the `w`

ay out of the loop: python while continuing: if found:
 ...found code... break else: ...advance... else: ...not-fou
 nd code... As a more concrete example, the following
 piece of code determines whether a positive integer
 num is prime—has no factors other than 1 and itself
 —by searching for factors greater than 1 (to run live,
 assign num before pasting): python x = num // 2 # 对
 某个 num > 1, 从一半开始while x > 1: if num % x ==
 0: # 余数为 0? 找到因子了print(num,'has factor', x) bre
 ak # 立即退出并跳过 else x -= 1 else: # 正常退出, 当 x
 到达 1 print(num,'is prime') Rather than setting a flag
 to be tested when the loop is exited, it inserts a brea
 k where a factor is found. This way, the loop else clau
 se can assume that it will be executed only if no facto
 r is found; if this code never hits the break, the numb
 er is prime. Trace through this code to see how this
 works. The loop else clause is also run if the body of t
 he loop is never executed, as you don't run a break in
 that event either; in a while loop, this happens if the t
 est in the header is false to begin with. Thus, in the p
 receding example you still get the "is prime" messag
 e if x is initially less than or equal to 1 (for instance, if
 num is 2). NOTE: Subprime code — This example det
 ermines primes, but only informally so. Numbers less
 than 2 are not considered prime by the strict mathem
 atical definition, but 1 and 0 are classified as such her
 e. To be really picky, this code also fails for negative
 numbers and succeeds for floating-point numbers wi

th all-zero decimal digits. Also note that its code must use `//` instead of `/` because we need the initial division to truncate remainders, not retain them. If you want to experiment with this code further, watch for its associated exercise at the end of Part IV, which wraps it in a function for reuse.

中文翻译

与循环 `else` 子句结合时, `break` 语句通常能消除其他语言中使用的“搜索状态标志”。抽象地说, 下面代码中的 `break` 在退出循环的路上会跳过 `else`: python while continuing : if found: ...found code... # 找到时的代码break else: ...advance... # 推进 (继续搜索) 的代码else: ...not-found code... # 未找到的代码举个更具体的例子, 下面的代码通过寻找大于 1 的因子, 判断正整数 `num` 是否为质数 (prime, 即除了 1 和它本身外没有其他因子) (要实际运行, 先给 `num` 赋值再粘贴): python x = num // 2 # 对某个 num > 1, 从一半开始while x > 1: if num % x == 0: # 余数为 0? 找到因子了print(num,'has factor', x) break # 立即退出并跳过 else x -= 1 else: # 正常退出, 当 x 到达 1 print(num,'is prime') 它没有设置一个退出循环后再测试的标志, 而是在找到因子的地方插入 `break`。这样一来, 循环 `else` 子句就可以假设: 它被执行, 当且仅当没有找到因子; 如果这段代码从未遇到 `break`, 那么这个数就是质数。亲手跟踪一遍这段代码, 看看它是如何工作的。循环 `else` 子句在循环体从未执行时也会运行——因为那种情况下你同样不会执行 `break`; 在 `while` 循环中, 当头行的测试一开始就为假时会发生这种情况。因此, 在上面的示例中, 如

果 x 初始时就小于等于 1（例如 num 是 2），你仍然会得到 "is prime" 的消息。注意：非严格质数代码（Subprime code）——这个示例判定质数，但只是非正式的。严格数学定义下小于 2 的数不是质数，但这里 1 和 0 会被判为质数。再挑剔一点：这段代码对负数会失败，对小数位全为零的浮点数会成功。另外注意它必须用 `//` 而不是 `/`，因为我们需要初始除法截断余数，而不是保留余数。如果你想进一步实验这段代码，留意第四部分末尾的相关练习——它会把这段代码包进函数以便复用。

代码分析

```
x = num // 2                # num > 1
while x > 1:
    if num % x == 0:        # 0
        print(num, 'has factor', x)
        break              # else
    x -= 1
else:                        # x 1
    print(num, 'is prime')
```

·解释器如何处理（以 $\text{num} = 9$ 为例）： $x = 4 \rightarrow$ 测试 $4 > 1$ 真 $\rightarrow 9 \% 4 == 1$ 非零（无因子） $\rightarrow x = 3 \rightarrow 9 \% 3 == 0$ 真 $\rightarrow$ 打印 9 has factor 3 $\rightarrow$ break 跳出，跳过 else，打印结束。再以 $\text{num} = 7$ 为例： x 从 3、2 一路减到 1，中途 $7 \% 3 == 1$ 、 $7 \% 2 == 1$ 全非零 $\rightarrow$ 没有 break，while $1 > 1$ 测试为假 $\rightarrow$ 循环正常结束 $\rightarrow$ 执行 else，打印 7 is prime。

·数学依据：一个合数 num 若存在大于 1 的因子，则必然存在不大于 $\text{num}/2$ 的因子，所以从 $\text{num} // 2$ 开始向下搜索足够了。`//` 是整除（向下取整），保证 x 是整数——

若用 / 会得到浮点数，% 的行为就变了。

- else 的语义核心：else 属于循环而不属于 if——它紧跟 while 的缩进级别。break 走的是"快速通道"，else 是"全程未中断"的奖励。

深度理解

- 核心概念：循环 else 是 Python 独有（与 Java/C++ 不同）的语法：循环正常结束（未 break）时执行一次。它是"搜索失败"场景的专门语法。

- 底层实现：编译期会把 else 块接到循环正常退出的分支上；break 则跳转到 else 块之后的地址。因此 break "绕过" else 是字节码级别的结构保证。

- 为什么这样设计：传统语言需要 found = False 标志 + 循环后 if not found: 两段代码。Python 用 else 把"未找到"的处理内联到循环结构里，省掉标志变量——这是"少一个状态、少一类 bug"的设计哲学。

- 实际问题：搜索（列表/数据库查询"没找到"）、校验（所有项通过才成功）、游戏（遍历完所有格子都没踩雷才加分）——一切"全遍历且未中断"才成立的逻辑。

- 初学者误区：① 误以为 else 是"循环正常走完的收尾工作"而滥用（普通收尾写循环外面即可）；② 把 else 缩进到 if 下面——那就变成 if-else 了，语义完全不同；③ 忘了"循环体一次都没执行"也会触发 else（书中特意提醒）

。

- 生活例子：else 像"图书馆找书"——翻遍整排书架都没找到（正常走完循环），才去服务台登记"未找到"；中途

找到了 (break) , 直接拿书走人, 不需要登记。

13.3.9 为什么需要循环 else? (Why the loop else ?)

英文原文

Because the loop else clause is unique to Python, it tends to perplex some newcomers (and even some veterans; in fact, a few either pointlessly code the loop else without a break or don't know that the loop else exists at all!). In general terms, the loop else simply provides explicit syntax for a common coding scenario—it is a coding structure that lets us catch the "other" way out of a loop, without setting and checking flags or conditions. Suppose, for instance, that we are writing a loop to search a list for a value and need to know whether the value was found after you exit the loop. We might code such a task this way (this code is intentionally abstract and incomplete; x is a sequence and match is a tester function to be defined):

```
python
found = False
while x and not found:
    if match(x[0]): # 前面的值?
        print('Found')
        found = True
    else:
        x = x[1:] # 切掉前面并重复
if not found:
    print('Not found')
```

Here, we initialize, set, and later test a found flag to determine whether the search succeeded or not. This is valid Python code, and it does work; however, this is exactly the sort of structure that the loop else clause is meant to handle. Here's an else equivalent:

```
python
while
```

```
x: # 当 x 为空时退出
if match(x[0]): print('Found') break
# 退出, 绕过
else x = x[1:] else: print('Not found') # 只有耗尽 x 时才到这里
```

This version is more concise. The `flag` is gone, and we've replaced the `if` test at the loop end with an `else` (lined up vertically with the word `while`). Because the `break` inside the main part of the `while` exits the loop and goes around the `else`, this serves as a more structured way to catch the search-failure case. Some readers might have noticed that the prior example's `else` clause could be replaced with a test for an empty `x` after the loop (e.g., `if not x:`). Although that's true in this example, the `else` provides explicit syntax for this coding pattern (it's more obviously a search-failure clause here), and such an explicit empty test may not apply in some cases. The loop `else` becomes even more useful when used in conjunction with the `for` loop—the topic of the next section—because sequence iteration is not under your control.

中文翻译

因为循环 `else` 子句是 Python 独有的，它常常让一些新手困惑（甚至让一些老手困惑；事实上，有少数人要么毫无意义地写了不带 `break` 的循环 `else`，要么根本不知道循环 `else` 的存在！）。一般来说，循环 `else` 只是为一种常见编码场景提供了显式语法——它是一种让我们抓住循环“另一种”出口的编码结构，而无需设置和检查标志或条件。例如，假设我们要写一个在列表中搜索值的循环，并且需要知道退出循环后值是否被找到。我们可能会这样编码（这段代码故意

写得抽象且不完整；x 是一个序列，match 是待定义的测试函数）：

```
python found = False while x and not found: if match(x[0]): # 前面的值? print('Found') found = True else: x = x[1:] # 切掉前面并重复if not found: print('Not found')
```

这里，我们初始化、设置、之后又测试了一个 found 标志，以判断搜索是否成功。这是合法的 Python 代码，也确实能工作；然而，这恰恰是循环 else 子句要处理的这类结构。下面是 else 等价版本：

```
python while x: # 当 x 为空时退出if match(x[0]): print('Found') break # 退出，绕过 else x = x[1:] else: print('Not found') # 只有耗尽 x 时才到这里这个版本更简洁。标志没了，我们用 else（与单词 while 垂直对齐）替换了循环末尾的 if 测试。因为 while 主体里的 break 退出循环并绕过 else，所以这是捕捉“搜索失败”情形的更有结构的方式。有些读者可能已经注意到，上一个示例的 else 子句可以用循环后的空 x 测试（例如 if not x:）来替代。虽然在这个示例里确实如此，但 else 为这种编码模式提供了显式语法（在这里它更明显地是一个“搜索失败”子句），而且这种显式的空测试在某些情况下可能并不适用。循环 else 与 for 循环（下一节的主题）结合时更有用——因为序列迭代不由你控制。
```

代码分析

```
found = False
while x and not found:
    if match(x[0]):          #
        print('Found')
        found = True
    else:
        x = x[1:]           #
if not found:
    print('Not found')
```

·传统写法的问题：① 状态变量 `found` 在三个地方出现（初始化、设置、测试）——任何一处写错都出 bug；② “正常耗尽 `x`”与“找到后退出”两种出口混在一起，退出后必须靠标志区分。

·对比 `else` 版本：`while x`: 只有两种出口——`break`（找到）或正常耗尽（未找到），`else` 恰好一一对应。状态被消除，退出语义自明。

·`if not x`: 替代方案的局限：本例可行，但若循环里还做了别的操作（如更新其他数据），退出后 `x` 为空不代表“未找到”；而且 `else` 的位置（紧跟循环）比“循环外的 `if`”更贴近逻辑本身——迭代不由你控制的 `for` 循环里，这个优势更明显。

深度理解

·核心概念：循环 `else` 消除“状态标志”模式，是 Python 对“结构化编程”的独特贡献。它与 `if/else` 的“否则”语义完全不同——它不是“条件不成立时”，而是“循环未中断时”。

·底层实现：两种写法在字节码层面其实都只是跳转 + 分

支；else 的价值不在性能，而在"让意图显式化"——读代码的人一眼就知道"这段只在未找到时执行"。

·为什么这样设计：Guido van Rossum 在设计 Python 时就加入了循环 else（受 ABC 语言影响），因为它让"搜索-未找到"这一高频模式有了第一公民语法，不必再造轮子。

·实际问题：任何"遍历+确认"逻辑：检查列表是否全合规、数据库查询是否命中、文件里是否含关键词、游戏中是否全部通关——else 都能省掉标志变量。

·初学者误区：① 不加 break 也写 else——此时 else 必然执行，等于白写（老手都犯的错）；② 把 else 缩进到 if 里导致逻辑反转；③ 与 if/else 混淆，以为 else 是"循环条件为假时执行"——那不是 else 的语义（测试为假 = 正常退出，else 反而会执行！）。

·生活例子：found 标志版像"出门前反复检查钥匙带了没、门锁了没"（多步确认）；else 版像"到车站才买票"——没买到（break 没发生）才需要处理，买到就直接走。

13.4 for 循环 (for Loops)

13.4.1 通用格式 (General Format)

英文原文

The for loop is a generic iterator in Python: it can step through the items in any ordered sequence or other iterable object. All told, the for statement works on

strings, lists, tuples, sets, dictionaries, and all other built-in iterables, as well as new user-defined objects that you'll learn how to create later with classes. We met for briefly in Chapter 4 and have used it in conjunction with sequence object types; let's expand on its usage more formally here. The Python for loop begins with a header line that specifies an assignment target (or targets), along with the object you want to step through. The header is followed by a block of (normally indented) statements that you want to repeat:

```
python for target in object: # 将对象的项赋给 target statements # 重复的循环体: 使用 target  
else: # 可选的 else statements # 若循环体没有以 break 退出则运行
```

When Python runs a for loop, it assigns the items in the iterable object to the target one by one and executes the loop body for each. The loop body typically uses the assignment target to refer to the current item in the sequence as though it were a cursor stepping through the sequence. While target can be any assignment target we met in Chapter 11, it's often just a simple name. This name is a possibly new variable that lives in the scope where the for statement itself is coded. There's not much unique about this name; it can even be changed inside the loop's body, but it will automatically be set to the next item in object when control returns to the top of the loop again. After the

loop this variable normally still refers to the last item visited, which is the last item in the sequence unless the loop exits early with a break statement. The for statement also supports an optional else block, which works exactly as it does in a while loop—it's executed if the loop exits without running into a break statement (i.e., if all items in the sequence have been visited). The break and continue statements introduced earlier also work the same in a for loop as they do in a while. Given all that, the for loop's complete format can be described this way:

```
python for target in object: # 将对象的项赋给 target statements
```

```
if test: break # 立即退出循环, 跳过 else if test: continue # 现在跳到循环顶部else: statements # 如果没有遇到'break' 则在退出时运行
```

中文翻译

for 循环是 Python 中的通用迭代器 (generic iterator) : 它可以遍历任何有序序列或其他可迭代对象中的项。总而言之, for 语句可以作用于字符串、列表、元组、集合、字典以及所有其他内置可迭代对象, 也能作用于你以后用类 (class) 创建的新用户自定义对象。我们在第 4 章简单见过 for, 并已配合序列对象类型使用过它; 现在让我们在这里更正式地展开它的用法。Python for 循环以头行开始, 头行指定一个 (或多个) 赋值目标 (assignment target) ,

以及你要遍历的对象。头行之后是一个（通常缩进的）语句块，即你想重复执行的循环体：`python for target in object: # 将对象的项赋给 target statements # 重复的循环体`；使用 `target else: # 可选的 else statements # 若循环体没有以 break 退出则运行` 当 Python 运行 `for` 循环时，它把可迭代 `object` 中的项逐一赋值给 `target`，并为每一项执行一次循环体。循环体通常把赋值目标当作“游标（cursor）”，用来指代序列中当前的项。虽然 `target` 可以是我们在第 11 章学过的任何赋值目标，但它通常只是一个简单的名字。这个名字是一个（可能是新的）变量，存在于 `for` 语句本身所在的作用域（scope）里。这个名字没什么特别之处；你甚至可以在循环体内改变它，但当控制流再次回到循环顶部时，它会被自动设置为 `object` 中的下一项。循环结束后，这个变量通常仍然指向最后访问的项——除非循环因 `break` 语句提前退出，否则它就是序列中的最后一项。`for` 语句还支持可选的 `else` 块，其工作方式与 `while` 循环中的完全一样——如果循环在没有遇到 `break` 语句的情况下退出（即序列中的所有项都被访问过），它就执行。前面介绍的 `break` 和 `continue` 语句在 `for` 循环中的工作方式与在 `while` 中相同。鉴于以上所有内容，`for` 循环的完整格式可以这样描述：`python for target in object: # 将对象的项赋给 target statements if test: break # 立即退出循环，跳过 else if test: continue # 现在跳到循环顶部 else: statements # 如果没有遇到 'break' 则在退出时运行`

深度理解

·核心概念：`for` 不是“计数器循环”，而是“逐个取项迭代器”。它的语义是赋值——`for target in object` 等价于“每

次迭代执行一次 `target = 下一项`。

·底层实现：`for` 基于迭代协议 (`iteration protocol`) 运行：解释器先对 `object` 调用 `iter()` 拿到迭代器，然后每轮调用 `next(iterator)` 取下一项，直到抛出 `StopIteration` 异常为止 (`for` 内部捕获它并结束循环)。这正是下一章的主题——`iter/next` 双方法协议。

·为什么这样设计：把“取下一个元素”抽象成协议而非语法细节，使得任何实现了 `iter/next` 的对象都能被 `for` 遍历——字符串、字典、文件、生成器、自定义类，一视同仁。这就是 Python “鸭子类型 (`duck typing`)”的体现。

·实际问题：遍历数据集合是程序最频繁的操作；`for` 让它无需手动维护索引变量 (`i`)、无需取长度、无需判断越界——省代码、省 `bug`。

·初学者误区：① 循环变量在循环结束后仍然存在（指向最后一项）——它不是“局部于循环”的，这在别的语言里常导致困惑；② 以为修改循环变量能影响迭代进度——不会，下一轮它会被自动覆盖；③ 想遍历索引却在遍历项（`for i in L` 得到的是项不是索引）。

·生活例子：`for` 像“流水线上的传送带”——物品一件件经过你面前，你只需处理当前这件 (`target`)，传送带负责“下一件”的调度，你不需要自己数件数。

13.4.2 基本用法 (Basic usage)

英文原文

Let's type a few for loops interactively now, so you can see how they are used in practice. As mentioned earlier, a for loop can step across any kind of sequence object. In our first example, for instance, we'll assign the name `x` to each of the three items in a list in turn, from left to right, and the print statement will be executed for each. Inside the print statement (the loop body), the name `x` refers to the current item in the list:

```
python >>> for x in ['app','script','program']: ... print(x, end=") ... app script program
```

The next two examples compute the sum and product of all the items in a list. Later in this chapter and later in this book you'll meet tools that apply operations such as `+` and `*` to items in a list automatically, but it's often just as easy to use a for:

```
python >>> sum = 0 >>> for x in [1, 2, 3, 4]: ... sum = sum + x ... >>> sum 10 >>> prod = 1 >>> for item in [1, 2, 3, 4]: prod = item ... >>> prod 24
```

中文翻译

现在让我们在交互式环境中敲几个 for 循环，看看它们在实际中如何使用。如前所述，for 循环可以遍历任何类型的序列对象。例如，在我们的第一个示例中，将名字 `x` 依次赋给列表中的三个项（从左到右），并为每个项执行一次 print 语句。在 print 语句（循环体）内部，名字 `x` 指代列表中的当前项：

```
python >>> for x in ['app','script','program']: ... print(x, end=") ... app script program
```

接下来的两个

示例计算列表中所有项的总和与乘积。在本章后面以及本书后面，你会遇到能自动把 + 和等操作应用到列表项上的工具，但很多时候直接用 for 同样简单：python >>> sum = 0 >>> for x in [1, 2, 3, 4]: ... sum = sum + x ... >>> sum 10 >>> prod = 1 >>> for item in [1, 2, 3, 4]: prod = item ... >>> prod 24

代码分析

```
>>> for x in ['app', 'script', 'program']:
...     print(x, end=' ')
...
app script program
```

·解释器如何处理：① 对列表调用 iter() 得到列表迭代器；② 第一轮 next() 返回'app'，赋值给 x，执行 print；③ 第二轮 next() 返回'script'.....直到 next() 抛出 StopIteration，for 静默捕获并结束。循环体只"看见"当前的 x，永远不知道自己在第几轮——除非你主动问。

·累加/累乘模式：sum = sum + x 是"累加器"模式 (accumulator) ——在循环体外初始化为幺元 (0 是加法幺元，1 是乘法幺元)，循环体内逐步累积。prod = item 是同一模式的简写。

·为什么用 0 和 1 初始化：0 + x = x、1 x = x，初始值不会污染结果。用 sum/prod 做变量名在交互式 REPL 里可行，但正式代码中它们会遮蔽内置函数 sum——这是本书早期版本的历史遗留习惯，现代写法是 total/product

。

深度理解

·核心概念：for 的三个标志性特性：自动取项、自动终止（项取完即停）、循环变量事后存活。

·底层实现：迭代器（iterator）持有"当前状态"——列表迭代器内部记着当前位置指针；字符串迭代器内部记着字符下标。next() 每次推进并返回；越界时抛 StopIteration。这保证了即使列表在循环中被修改，迭代器也按自己的节奏走（这正是 13.5.7 节"修改列表陷阱"的根源）。

·为什么这样设计：把"遍历"从程序员手里收走，交给解释器统一优化——CPython 的 for 比等价的 while + 下标通常更快，因为迭代在 C 层面完成。

·实际问题：求和、求积、统计、过滤、转换——一切"对集合里的每个元素做点什么"的任务都是 for 的主场。

·初学者误区：① 把累加器放在循环体内重新初始化（结果永远是最后一个元素）；② 混淆"修改循环变量"与"修改列表"（见 13.5.7）；③ 循环变量名与外部变量重名导致意外覆盖。

·生活例子：累加器像"收银员的小票金额"——每扫一件商品就加一次，从 0 开始；循环变量像"正在扫码的商品"，扫完就换下一件。

13.4.3 其他数据类型 (Other data types)

英文原文

Any sequence works in a for, as it's a generic tool. For example, for loops also work on strings and tuples:

```
python >>> S='Python'>>> T=('web','num','app')
>>> for x in S: print(x, end='') # 迭代一个字符串... P y
t h o n >>> for x in T: print(x, end='') # 迭代一个元组..
. web num app
```

In fact, as we'll explore in the next chapter when we formalize the notion of iterables, for loops can even work on some objects that are not sequences—including files.

中文翻译

任何序列都能用于 for，因为它是一个通用工具。例如，for 循环同样适用于字符串和元组：

```
python >>> S='Python'
>>> T=('web','num','app') >>> for x in S: print(x, en
d='') # 迭代一个字符串... P y t h o n >>> for x in T: pri
nt(x, end='') # 迭代一个元组... web num app
```

事实上，正如我们在下一章正式定义“可迭代（iterable）”概念时要探讨的，for 循环甚至能作用于一些不是序列的对象——包括文件（file）。

代码分析


```
>>> S = 'Python'
>>> T = ('web', 'num', 'app')
>>> for x in S: print(x, end=' ')    #
...
P y t h o n
>>> for x in T: print(x, end=' ')    #
...
web num app
```

·解释器如何处理：字符串的迭代器按字符产出（'Python' 产出 6 个字符）；元组的迭代器按元素产出（3 个字符串）。for 对它们一视同仁——它不关心对象是"什么类型"，只关心"能不能迭代"。

·关键预告：文件不是序列（不支持下标索引），但它实现了迭代协议——每轮 next() 返回一行。所以 for line in open('f.txt') 逐行读文件，这是本章末尾"文件扫描器"部分的核心技巧。

·"通用工具"的含义：统一接口带来统一心智模型：不管对象内部结构如何，for x in obj 永远表达"取遍所有项"。

深度理解

·核心概念：for 的普适性来自迭代协议而非序列类型。字符串/元组/列表/字典/集合/文件/生成器，全都能被 for 遍历。

·底层实现：字符串迭代器逐字符；字节串逐字节；字典迭代器逐键；文件迭代器逐行——每种类型自己决定"项"是什么。iter() 返回的对象不同，next() 的语义不同，但 for 的骨架完全不变。

·为什么这样设计：迭代协议把"遍历策略"下放到每个类型

内部，语言核心只保留统一入口。新增一个容器类型只要实现 `iter/next`，立刻获得全部迭代能力——这是开放-封闭原则（OCP）的经典实践。

·实际问题：遍历 CSV 行、逐行处理日志、逐块读取大文件——文件迭代器让“处理大文件”不再依赖一次性读入内存。

·初学者误区：① 以为 `for` 只能遍历数字范围或列表；② 试图对文件用 `for i in range(len(f))`——文件不可索引；③ 忘记字典 `for` 默认遍历的是键而不是键值对。

·生活例子：不同容器的迭代器像不同型号的“取料机”——超市货架（列表）按位置取、糖罐（字典）按标签取、流水线（文件）按顺序吐出一件件——但操作工（`for`）的操作方式始终一样。

13.4.4 `for` 循环中的元组（序列）赋值（Tuple (sequence) assignment in `for` loops）

英文原文

If you're iterating through a sequence of tuples, the loop target itself can actually be a tuple of targets. This is just another case of the tuple-unpacking assignment we studied in Chapter 11 at work. Remember, the `for` loop assigns items in the sequence object to the target, and assignment works the same everywhere:

```
python >> T = [(1, 2), (3, 4), (5, 6)] >> for (a, b) in T:  
# 元组赋值大显身手  
... print(a, b) ... 1 2 3 4 5 6
```

Here, the first time through the loop is like running $(a, b) = (1, 2)$, the second time is like $(a, b) = (3, 4)$, and so on. The net effect is to automatically unpack the current tuple on each iteration. List syntax works as a for target too because tuple and list assignment are both sequence assignment, and tuple parentheses are optional:

```
python >> for [a, b] in T: # 列表赋值: 同样的效果 >> f  
or a, b in T: # 不带括号的元组: 同样的效果
```

This list-of-tuples data format is commonly used in conjunction with the `zip` call you'll meet later in this chapter, to implement parallel traversals. It also crops up in conjunction with SQL databases in Python, where query result tables are returned as sequences of sequences like the list used here—the outer list is the database table, the nested tuples are the rows within the table, and tuple (i.e., sequence) assignment extracts columns.

As we've seen in earlier chapters, tuples in for loops also come in handy to iterate through both keys and values in dictionaries using the `items` method, rather than looping through the keys and indexing to fetch the values manually:

```
python >> D = {'a': 1, 'b': 2} >> for key in D:
```

```
... print(key, '=', D[key]) # 使用字典迭代器和索引... a  
=> 1 b => 2 >> list(D.items()) [('a', 1), ('b', 2)] >> for  
(key, value) in D.items():
```

```
... print(key, '=', value) # 同时迭代键和值... a => 1 b =  
> 2 It's important to note that tuple assignment in fo  
r loops isn't a special case; any assignment target wo  
rks syntactically after the word for. For example, we c  
an always assign manually within the loop to unpack:
```

```
python >> T [(1, 2), (3, 4), (5, 6)] >> for both in T:
```

```
... a, b = both # 手动赋值等价物... print(a, b) ... 1 2 3 4  
5 6
```

But tuples in the loop header save us an extra step when iterating through sequences of sequences. As suggested in Chapter 11, even nested structures may be automatically unpacked this way in a for:

```
python >> ((a, b), c) = ((1, 2), 3) # 嵌套序列也可以 >>  
a, b, c (1, 2, 3) >> for ((a, b), c) in [((1, 2), 3), ((4, 5), 6)  
]: print(a, b, c)
```

```
... 1 2 3 4 5 6
```

Even this is not a special case, though—the for loop simply runs the sort of assignment we ran just before it, on each iteration. Any nested sequence structure may be unpacked this way, simply because sequence assignment is so generic:

```
python >> for ((a, b), c) in [[[1, 2], 3), ['XY', 6]]: print(a, b, c)
... 1 2 3 X Y 6
```

中文翻译

如果你遍历的是元组的序列，那么循环目标本身实际上可以是一个元组的目标。这只是我们在第 11 章学过的元组解包赋值 (tuple-unpacking assignment) 在工作。记住，for 循环把序列对象中的项赋值给目标，而赋值在任何地方的工作方式都一样：python >>> T = [(1, 2), (3, 4), (5, 6)] >>> for (a, b) in T: # 元组赋值大显身手... print(a, b) ... 1 2 3 4 5 6 这里，循环第一次就像执行 (a, b) = (1, 2)，第二次就像执行 (a, b) = (3, 4)，依此类推。净效果是每次迭代自动解包 (unpack) 当前的元组。列表语法同样可以作为 for 目标，因为元组赋值和列表赋值都是序列赋值 (sequence assignment)，而且元组括号是可选的：python >>> for [a, b] in T: # 列表赋值：同样的效果 >>> for a, b in T: # 不带括号的元组：同样的效果 这种“元组列表”的数据格式常与本章后面会遇到的 zip 调用配合使用，以实现并行遍历。它在 Python 的 SQL 数据库场景中也很常见：查询结果表以“序列的序列”形式返回，就像这里用的列表——外层列表是数据库表，嵌套元组是表中的行，元组（即序列）赋值则提取出各列。正如我们在前面章节所见，for 循环中的元组还有一个妙用：用 items 方法同时遍历字典的键和值，而不是循环遍历键再手动索引取值：python >>> D = {'a': 1, 'b': 2} >>> for key in D: ... print(key, '=', D[key]) # 使用字典键迭代器和索引... a => 1 b => 2 >>> li

```

st(D.items()) [('a', 1), ('b', 2)] >>> for (key, value) in D.
items(): ... print(key, '=', value) # 同时迭代键和值... a =
> 1 b => 2 重要的是要注意：for 循环中的元组赋值不是
特殊情形；任何赋值目标在单词 for 之后语法上都有效。例
如，我们总可以在循环内部手动赋值来解包：python >>>
T [(1, 2), (3, 4), (5, 6)] >>> for both in T: ... a, b = both
# 手动赋值等价物... print(a, b) ... 1 2 3 4 5 6 但在遍历"序
列的序列"时，循环头里的元组能帮我们省掉这一步。正如
第 11 章所提示的，即使是嵌套结构也能用这种方式在 for
中自动解包：python >>> ((a, b), c) = ((1, 2), 3) # 嵌套
序列也可以>>> a, b, c (1, 2, 3) >>> for ((a, b), c) in [((
1, 2), 3), ((4, 5), 6)]: print(a, b, c) ... 1 2 3 4 5 6 不过，即
使这样也不是特殊情形——for 循环只是在每次迭代时运行
一次我们刚才运行过的赋值而已。任何嵌套序列结构都可以
这样解包，仅仅因为序列赋值是如此通用：python >>> f
or ((a, b), c) in [([1, 2], 3), ['XY', 6]]: print(a, b, c) ... 1 2
3 X Y 6

```

代码分析

```

>>> T = [(1, 2), (3, 4), (5, 6)]
>>> for (a, b) in T:                                #
...     print(a, b)
...

```

·解释器如何处理：每次迭代先由迭代器产出整个元组（如 (1, 2)），然后执行一次元组解包赋值 (a, b) = (1, 2)：按位置把 1 绑到 a、2 绑到 b。这正是第 11 章序列赋值机制的复用——for 的 target 位置就是赋值语句的左值位置，语法规则完全通用。

·统一性洞察：for [a, b] in T 和 for a, b in T 之所以等价，是因为列表赋值与元组赋值在语义上都是序列赋值——目标形态可以任意变换。甚至 `[(1, 2), 3], ['XY', 6]` 里的第二个元素是列表不是元组，也能解包——序列赋值不挑剔容器类型，只按位置配对。

·字典解包：`D.items()` 返回键值对视图，迭代产出 `('a', 1)` 这类二元元组，`(key, value)` 直接解包。相比 `for key in D + D[key]` 索引取值，少一次哈希查找，写法也更直白。

深度理解

·核心概念：for 的目标是"任何赋值目标"——简单名、元组、列表、嵌套结构、带星号目标都行。这归功于序列赋值的通用性。

·底层实现：for 的赋值步骤与普通赋值语句共享同一套字节码机制（`UNPACKSEQUENCE` 指令处理解包），所以"能用 `=` 的地方就能用 for 目标"。

·为什么这样设计：数据库行、坐标对、键值对、CSV 记录——现实数据天然就是"结构化的序列"。让 for 直接解包，避免了"取项后再手动拆分"的样板代码。

·实际问题：遍历数据库查询结果（每行解包为列）、遍历 zip 配对结果、遍历字典键值对、解析 CSV/表格数据。

·初学者误区：① 目标元素数量与项长度不匹配——多元素/少元素都会抛 `ValueError`（解包失败）；② 忘了括号可省略（`for a, b in T` 合法）；③ 以为这是 for 的特殊语法而没意识到它只是通用赋值。

·生活例子：像"整箱货物开箱登记"——for 把整箱（元组）搬来，目标 (a, b) 自动拆成两件分别入库；嵌套目标 ((a,b), c) 则是"箱中还有盒"。

13.4.5 for 循环中的扩展解包赋值 (Extended-unpacking assignment in for loops)

英文原文

In fact, because the loop variable in a for loop can be any assignment target, we can also use the starred names and other targets of extended-unpacking assignment here to extract both items and sections of sequences within sequences. Because this works in assignment statements, it automatically works in for loops too.

This topic was introduced in Chapter 11, but here's a quick refresher to reinforce the technique. Consider the tuple assignment form introduced in the prior section. A tuple of values is assigned to a tuple of names on each iteration, exactly like a simple assignment statement:

```
python >> a, b, c = (1, 2, 3) # 元组赋值 >> a, b, c (1, 2, 3) >> for (a, b, c) in [(1, 2, 3), (4, 5, 6)]: # 用于 for 循环
... print(a, b, c) ... 1 2 3 4 5 6
```


Because sequence assignment supports a more general set of names with a starred target to collect multiple items, we can use the same syntax to extract parts of nested sequences in the for loop:

```
python >> a, b, c = (1, 2, 3, 4) # 扩展解包赋值 >> a, b, c (1, [2, 3], 4) >> for (a, b, c) in [(1, 2, 3, 4), (5, 6, 7, 8)]:  
:  
... print(a, b, c) ... 1 [2, 3] 4 5 [6, 7] 8
```

In practice, this approach might be used to pick out multiple columns from rows of data represented as nested sequences. As usual in Python, you can achieve similar effects with more basic tools—in this case by slicing. The only difference is that slicing returns a type-specific result, whereas starred targets always receive lists:

```
python >> for all in [(1, 2, 3, 4), (5, 6, 7, 8)]: # 手动切片版本  
... a, b, c = all[0], all[1:-1], all[-1] ... print(a, b, c) ... 1 (2, 3) 4 5 (6, 7) 8
```

Finally, all the starred-target forms work in for as in = assignment statements, including nested sequences, indexes, and slices (though practical roles for code like the following are probably much more rare than common!):

```
python >> L, M = [1, 2], [3, 4] >> pairs = [(5, 6), (7, 8), (9, 10)]  
2 >> for [(a, X), (b, L[0]), (c, M[:0])] in pairs:
```

```
... print(f'<{a=} {X=}> <{b=} {L=}> <{c=} {M=}>') ... <
a=5 X=[6]> <b=7 L=[[8], 2]> <c=9 M=[10, 3, 4]> <a
=5 X=[6]> <b=7 L=[[8], 2]> <c=9 M=[10, 10, 3, 4]>
```

See Chapter 11 for more on the extended-unpacking form of assignment.

中文翻译

事实上，因为 for 循环的循环变量可以是任何赋值目标，我们也可以在这里使用带星号的名字等扩展解包赋值（extended-unpacking assignment）目标，来提取序列中序列的项和片段。因为它在赋值语句中有效，所以也自动适用于 for 循环。这个主题在第 11 章介绍过，这里快速复习一下以强化这项技巧。考虑上一节介绍的元组赋值形式：每次迭代把一个值元组赋给一个名字元组，和普通赋值语句一模一样：python >>> a, b, c = (1, 2, 3) # 元组赋值>>> a, b, c (1, 2, 3) >>> for (a, b, c) in [(1, 2, 3), (4, 5, 6)]: # 用于 for 循环... print(a, b, c) ... 1 2 3 4 5 6 因为序列赋值支持更通用的名字集合——用带星号的目标收集多个项——我们可以用同样的语法在 for 循环中提取嵌套序列的各个部分：python >>> a, b, c = (1, 2, 3, 4) # 扩展解包赋值>>> a, b, c (1, [2, 3], 4) >>> for (a, b, c) in [(1, 2, 3, 4), (5, 6, 7, 8)]: ... print(a, b, c) ... 1 [2, 3] 4 5 [6, 7] 8 在实践中，这种方法可以用来从嵌套序列表示的数据行中挑出多个列。和 Python 中常见的情况一样，你也可以用更基础的工具达到类似效果——这里就是切片。唯一的区别是：切片返回类型特定的结果，而带星号的目标总是收到列表：python >>> for all in [(1, 2, 3, 4), (5, 6, 7, 8)]: # 手动切片版本... a, b, c = all[0], all[1:-1], all[-1] ... print(a, b, c) ... 1 (2, 3

) 4 5 (6, 7) 8 最后，所有带星号目标的形式在 for 中与在 = 赋值语句中一样有效，包括嵌套序列、索引和切片（虽然下面这类代码的实际用途可能远比它的出场率少！）：python >>> L, M = [1, 2], [3, 4] >>> pairs = [(5, 6), (7, 8), (9, 10)] 2 >>> for [(a, X), (b, L[0]), (c, M[:0])] in pairs: ... print(f'<{a=} {X=}> <{b=} {L=}> <{c=} {M=}>') ... <a=5 X=[6]> <b=7 L=[[8], 2]> <c=9 M=[10, 3, 4]> <a=5 X=[6]> <b=7 L=[[8], 2]> <c=9 M=[10, 10, 3, 4]>关于扩展解包赋值的更多内容，参见第 11 章。

代码分析

```
>>> a, *b, c = (1, 2, 3, 4)          #
>>> a, b, c
(1, [2, 3], 4)
```

·解释器如何处理：带星号的目标由 UNPACKEX 字节码处理——它先按左右两端固定位置取首尾（a 取 1、c 取 4），剩下的中间部分收集成一个列表赋给 b（即 [2, 3]）。注意 b 的类型是列表，不是元组——这是扩展解包的固定规则。

·切片版对比：all[1:-1] 返回与 all 同类型的结果（元组切片得元组 (2, 3)），而星号目标恒定返回列表 [2, 3]。两版打印结果仅此一个类型差异。

·怪例拆解：pairs = [(5,6),(7,8),(9,10)] 2 创建两个共享同一个内层列表的副本（2 复制的是引用）；目标 [(a, X), (b, L[0]), (c, M[:0])] 每一层都在解包：首元素 (5, 6) 解出 a=5, X=[6]；L[0] 在赋值前取值 [1, 2]，解出 b=7 并让 L[0] 收集空列表 []——等等，输出是 L=[[8], 2]。真相是：

L[0] 把 (7, 8) 的剩余部分 [8] 收集后重新赋值给 L[0]！第一轮后 L[0] 变成 [8]，所以第二轮 (b, L[0]) 用新的 L[0]=[8] 解包 (9, 10) → b=9、L[0] 被赋为 [10]，于是打印 M=[10, 10, 3, 4] 时 L=[[8], 2] 不变但 c 与 M 变化。这就是"赋值目标里写索引"的危险之处——目标先求值索引、赋值时再写入，副作用会跨轮累积。

·作者的态度：这个例子纯粹展示"语法上可行"，作者明说实际用途"远比出场率少"——教学价值在于证明 for 目标 = 通用赋值目标，而不是鼓励这样写。

深度理解

·核心概念：星号 (*) 目标在 for 里同样可用，用于"取首尾、收中间"或"取头收尾"式解包。

·底层实现：扩展解包依赖 UNPACKEX 字节码；赋值目标里的索引/切片（如 L[0]）走 STORESUBSCR/STORESLICE 指令——先计算下标再存储，所以会产生"修改数据源"的副作用。

·为什么这样设计：把"解包能力"做成赋值语句的通用属性，for 只是顺带受益——语言设计上的一次性投资，多处复用。

·实际问题：从固定格式的数据行中提取首列、末列与中段；从变长记录中"掐头去尾"；API 返回 (status, data) 时拆分。真实项目中，for a, rest in rows 是高频写法。

·初学者误区：① 以为星号目标得到的是元组——永远是列表；② 一个解包目标里不能出现两个星号 (a, b, c = x 语法错误)；③ 在目标里混入索引/切片表达式，以为"只

是解包"——它可能改写源数据结构。

·生活例子：星号解包像"自助餐分盘"——b 就是"把中间那些菜都装进这个盘"，具体几样不关心；往目标里塞索引则是"装盘时还顺便改标签"——能行但容易乱。

13.4.6 嵌套 for 循环 (Nested for loops)

英文原文

Now let's look at some for loops that are a bit more sophisticated than those demoed so far. The first shows what happens when for loops are nested—the inner loop is run for every iteration of the outer loop, and the + within the inner loop combines their items by using each loop's variable:

```
python >>> for x in 'abc': # 对于第一个字符串中的每一项...
for y in '123': # 对于另一个字符串中的每一项... print(
x + y, end='') # 拼接两者的当前项... a1 a2 a3 b1 b2 b3
c1 c2 c3
```

The next example kicks the nesting up a notch, illustrating both three-level statement nesting and the loop else clause in a for. Given a list of objects (items) and a list of keys (tests), this code searches for each key in the objects list and reports on the search's outcome:

```
python >>> items = ['aaa', 111, (4, 5), 2.01] # 对象列表
>>> tests = [(4, 5), 3.14] # 要搜索的键
>>> >>> for
```

```
key in tests: # 对于所有键... for item in items: # 对于所有项... if item == key: # 检查是否匹配... print(key,'was found') ... break ... else: ... print(key,'not found!') ... (4, 5) was found 3.14 not found!
```

Because the nested if runs a break when a match is found, the inner loop's else clause can assume that if it is reached, the search has failed. Notice the nesting here. When this code runs, there are two loops going at the same time: the outer loop scans the keys list, and the inner loop scans the items list for each key. The nesting of the loop else clause is critical; it's indented to the same level as the header line of the inner for loop, so it's associated with the inner loop, not the if or the outer for. The preceding example is illustrative, but it may be easier to code if we employ the in operator to test membership. Because in implicitly scans an object looking for a match (at least logically), it replaces the inner loop:

```
python >>> for key in tests: # 对于所有键... if key in items: # 让 Python 检查匹配... print(key,'was found') ... else: ... print(key,'not found!') ... (4, 5) was found 3.14 not found!
```

In general, it's a good idea to let Python do as much of the work as possible (as in this solution) for the sake of both brevity and performance. Our final example is similar, but builds a list as it goes for later use instead of printing. It performs a typical data-structure t

ask with a `for`—collecting common items in two sequences (it's nearly intersection, unless there are duplicate values). After the loop runs, `res` refers to a list that contains all the items found in `seq1` and `seq2`:

```
python >>> seq1 = 'trippy'>>> seq2 = 'python'>>>
>>> res = [] # 从空开始>>> for x in seq1: # 扫描第一个序列...
if x in seq2: # 共同的项? ... res.append(x) # 添加到结果末尾...
>>> res ['t','p','p','y']
```

Unfortunately, this code is equipped to work only on two specific variables: `seq1` and `seq2`. It would be nice if this loop could somehow be generalized into a tool you could use more than once. As you'll see, that simple idea leads us to functions, the topic of the next part of the book. Of course, if you read Chapter 4 or Chapter 5, you know that Python has sets that provide true intersection with the `&` operator—but the result's order is scrambled, duplicates are dropped, and multiple conversions are required to match:

```
python >>> list(set(seq1) & set(seq2)) # 用集合求真正的交集['p','y','t']
```

More usefully, this code also exhibits the classic list comprehension pattern—collecting a results list with a `for` iteration and optional filter test—and could be coded much more concisely with this tool:

```
python >>> [x for x in seq1 if x in seq2] # 让 Python 收集结果['t','p','p','y']
```

But you'll have to read on to the next chapter for the rest of this story.

中文翻译

现在让我们看一些比之前演示的更复杂的 for 循环。第一个展示 for 循环嵌套时会发生什么——内层循环在外层循环的每次迭代中都会完整运行一遍，内层循环里的 + 用两个循环各自的变量拼接它们的项：python >>> for x in 'abc': # 对于第一个字符串中的每一项... for y in '123': # 对于另一个字符串中的每一项... print(x + y, end="") # 拼接两者的当前项... a1 a2 a3 b1 b2 b3 c1 c2 c3 下一个示例把嵌套再升一级，同时展示了三层语句嵌套和 for 中的循环 else 子句。给定一个对象列表 (items) 和一个键列表 (tests)，这段代码在对象列表中搜索每个键，并报告搜索结果：python >>> items = ['aaa', 111, (4, 5), 2.01] # 对象列表>>> tests = [(4, 5), 3.14] # 要搜索的键>>> >>> for key in tests: # 对于所有键... for item in items: # 对于所有项... if item == key: # 检查是否匹配... print(key, 'was found') ... break ... else: ... print(key, 'not found!') ... (4, 5) was found 3.14 not found! 因为嵌套的 if 在找到匹配时执行 break，所以内层循环的 else 子句可以假定：如果它被执行到了，就说明搜索失败了。注意这里的嵌套关系。这段代码运行时有两个循环同时进行：外层循环扫描键列表，内层循环为每个键扫描项列表。循环 else 子句的嵌套位置至关重要：它缩进与内层 for 循环头行相同的级别，因此它属于内层循环，而不是 if 或外层 for。上面的示例很有启发性，但如果用 in 运算符测试成员关系，编码会更容易。因为 in 会（至少在逻辑上）隐式扫描对象寻找匹配，它

替代了内层循环：python >>> for key in tests: # 对于所有键... if key in items: # 让 Python 检查匹配... print(key, 'was found') ... else: ... print(key, 'not found!') ... (4, 5) was found 3.14 not found! 一般来说，让 Python 尽可能多地干活是个好主意（就像这个解决方案），既简洁又有性能。我们最后的示例类似，但它不打印，而是边循环边构建一个列表供以后使用。它用 for 完成一个典型的数据结构任务——收集两个序列中的共同项（这几乎就是交集（intersection），除非有重复值）。循环结束后，res 指向一个列表，包含所有在 seq1 和 seq2 中都出现的项：python >>> seq1 = 'trippy'>>> seq2 = 'python'>>> >>> res = [] # 从空开始>>> for x in seq1: # 扫描第一个序列... if x in seq2: # 共同的项? ... res.append(x) # 添加到结果末尾...>>> res ['t','p','p','y'] 遗憾的是，这段代码只能作用于两个特定变量：seq1 和 seq2。要是这个循环能推广成一个可以多次使用的工具就好了。如你所见，这个简单的想法会引领我们走向函数（functions）——本书下一部分的主题。当然，如果你读过第 4 章或第 5 章，你会知道 Python 的集合（set）能用 & 运算符提供真正的交集——但结果顺序是打乱的、重复项被去掉，而且需要多次转换才能对上：python >>> list(set(seq1) & set(seq2)) # 用集合求真正的交集['p','y','t'] 更有用的是，这段代码还展示了经典的列表推导（list comprehension）模式——用一次迭代和可选的过滤测试来收集结果列表——用这个工具编码会简洁得多：python >>> [x for x in seq1 if x in seq2] # 让 Python 收集结果['t','p','p','y'] 不过，这个故事的其余部分，你得接着读下一章才知道。

代码分析

```
>>> for x in 'abc':                #
...     for y in '123':            #
...         print(x + y, end=' ') #
...
a1 a2 a3 b1 b2 b3 c1 c2 c3
```

· 解释器如何处理：外层迭代器产出'a'后，内层循环完整跑完'123' 三轮（输出 a1 a2 a3），内层结束后回到外层取'b'，再启动新一轮内层.....这就是"内层每轮 × 外层每轮"的笛卡尔积（cartesian product）：共 $3 \times 3 = 9$ 个输出。

· for-else 搜索：break 只中断内层循环。内层 else 的缩进与内层 for 对齐——归属内层。(4, 5) 在 items 中找到（第三项），内层 break，跳过内层 else；3.14 找遍四项都没有，内层正常耗尽，执行内层 else 打印 "not found!"。外层循环对每个键负责调用一次内层。

· in 运算符替代内层循环：key in items 在 CPython 内部就是一次循环（列表用 PyObjectRichCompareBool 逐个比较），它把"手工内层循环"压缩成一次 C 级扫描——更短、更快，这就是作者说的"让 Python 干活"。

· 交集收集：'trippy' 与 'python' 的共同字符是 t,p,y，但手动循环保留顺序与重复（'p' 出现两次），结果 ['t','p','p','y']——注意这不是数学意义上的"集合交集"（集合版本 ['p','y','t'] 去重且乱序）。res.append(x) 是"边遍历边构建列表"模式。

深度理解

·核心概念：嵌套循环 = 外层每轮驱动内层完整一轮，形成"组合遍历"；循环 else 嵌套时必须注意归属；能用内置运算符 (in、集合 &) 替代手写循环时优先用。

·底层实现：in 运算符对列表是线性扫描 ($O(n)$)，对集合/字典是哈希查找 ($O(1)$)；& 集合交集在 C 层用哈希表实现。手写 Python 循环逐元素比较是最慢的——这是"性能"考虑的第一课。

·为什么这样设计：Python 把"扫描找匹配"封装进 in，把"交集"封装进集合，让程序员表达意图而非步骤——这就是"显式优于隐式"的平衡点：语言替你把通用步骤包好。

·实际问题：矩阵遍历、组合生成、数据库行匹配、去重交集、集合运算——嵌套循环和内置容器运算符各有所长，按数据量选择。

·初学者误区：① else 缩进错误导致"找到也说没找到"；② 忘记 break 只跳内层，外层照跑；③ 用三重以上嵌套循环处理大数据——先想想哈希表或推导式。

·生活例子：嵌套循环像"两本电话簿交叉核对"——每翻开外簿一个人名，就要把内簿从头翻到尾。in 运算符像"把内簿做成索引卡"——直接翻到对应字母。

13.5 循环编码技巧 (Loop Coding Techniques)

13.5.1 总览 (Overview)

英文原文

The for loop we just studied subsumes most counter-style loops. It's generally simpler to code and often quicker to run than a while, so it's the first tool you should reach for whenever you need to step through a sequence or other iterable. In fact, as a general rule, you should resist the temptation to count things in Python — its iteration tools automate much of the work you do to loop over collections in lower-level languages like C. Still, there are situations where you will need to iterate in more specialized ways. For example, what if you need to visit every second or third item in a list, or change the list along the way? How about traversing more than one sequence in parallel, in the same for loop? What if you need indexes too? You can always code such unique iterations with a while loop and manual indexing, but Python provides a set of built-ins that allow you to specialize the iteration in a for:- The built-in range function produces a series of successively higher integers, which can be used as indexes in a for.- The built-in zip function returns a series of parallel-item tuples, which can be used to traverse multiple sequences in a for.- The built-in enumerate function generates both the values and indexes of items in an iterable, so we don't need to count manually. Because for loops may run quicker than while-bas

ed counter loops, it's to your advantage to use tools like these that allow you to use for whenever possible. Let's look at each of these built-ins in turn, in the context of common roles. As you'll see, some loop-coding alternatives are more valid than others.

中文翻译

我们刚学过的 for 循环涵盖了大多数计数器式循环。它通常比 while 编码更简单、运行也往往更快，所以每当需要遍历序列或其他可迭代对象时，它应该是你第一个想到的工具。事实上，作为一般规则，你应该抵制在 Python 中计数的诱惑——它的迭代工具自动化了你在 C 这类底层语言中为遍历集合所做的大部分工作。不过，确实有些情况需要以更特殊的方式迭代。例如，如果你需要访问列表中的每隔一个或第三个项呢？或者一边遍历一边修改列表呢？在同一 for 循环中并行遍历多个序列呢？如果你还需要索引呢？你总是可以用 while 循环加手动索引来编码这些特殊迭代，但 Python 提供了一组内置函数，让你能对 for 的迭代进行特化：

- 内置 range 函数产生一系列递增的整数，可以在 for 中用作索引。
- 内置 zip 函数返回一系列"并行项"元组，可用于在 for 中遍历多个序列。
- 内置 enumerate 函数同时生成可迭代对象中项的值和索引，这样我们就不必手动计数了。

因为 for 循环可能运行得比 while 计数器循环快，所以尽量使用这些让你能用 for 的工具对你有利。让我们在常见角色的上下文中，逐一看看这些内置函数。你将看到，有些循环编码的替代方案比其他方案更可取。

深度理解

- 核心概念：循环编码技巧的本质是"把计数交给内置函数"——range（要索引）、zip（要并行）、enumerate（索引+值都要），从而始终保住 for 的速度与简洁。
- 底层实现：这三个都是惰性（lazy）迭代器——不立即构建完整结果，而是每轮生成一个（这就是下一章的核心概念，也是 range(109) 不占内存的原因）。CPython 的 for 直接驱动它们的 next，快于纯 Python 的 while 索引循环。
- 为什么这样设计：ex-C 程序员的思维惯性是"自己维护计数器"，但 Python 认为计数是"重复性的细节"，应交给语言自动化——这是"抵抗计数的诱惑"这句话的由来。
- 实际问题：大数据集上遍历不能预生成列表；多序列配对；带行号处理数据——三种需求，三个内置，一条 for。
- 初学者误区：拿 while 当"更正统的循环"；在 for 里用 range 却不用 enumerate/zip，费力不讨好。
- 生活例子：range 像"编号叫号"，zip 像"男女配对站队"，enumerate 像"带号签到"——三样工具各有分工，但共同点是"不用你自己数"。

13.5.2 计数器循环：range（Counter Loops: range）

英文原文

Our first loop-related function, `range`, is a general tool that can be used in a variety of contexts. We met it briefly in Chapter 4 and have used it occasionally along the way. Although it's used often to generate indexes in a loop, you can call it anywhere you need a series of integers (see Chapter 11's enumerated-names trick for a prime example).

As we've seen, `range` is an iterable that generates items on demand, so we need to wrap it in a list call to display all its results at once in a REPL. Surprisingly, `range`'s results support some sequence operations, but not all; per Chapter 9, it was reclassified in Python's docs as a sort of sequence object type, though one with less functionality than lists and tuples—and much less basis in reality:

```
python >> list(range(5)), list(range(2, 5)), list(range(0, 10, 2)) ([0, 1, 2, 3, 4], [2, 3, 4], [0, 2, 4, 6, 8]) >> range(5)[2], range(5)[1:3], list(range(5)) + [6, 7] (2, range(1, 3), [0, 1, 2, 3, 4, 6, 7]) >> range(5) + [6, 7] TypeError: unsupported operand type(s) for +: 'range' and 'list'
```

Categorization aside, `range` usage is straightforward. With one argument, `range` generates a series of integers from zero up to but not including the argument's value. If you pass in two arguments, the first is taken as the lower bound. And an optional third argument can give a step; if it is used, Python adds the step to

each successive integer in the result (the step defaults to +1). Ranges can also be nonpositive and nonascending, if you need them to be:

```
python >> list(range(-5, 5)) [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4]
>> list(range(5, -5, -1)) [5, 4, 3, 2, 1, 0, -1, -2, -3, -4]
```

We'll take a deeper look at iterables in Chapter 14. In this case, Python 2.X had an optimized built-in named `xrange`, which was like its `range` but didn't build a result list in memory all at once; which was later superseded in 3.X by the generator behavior of its `range`; which was later rebranded a sequence by 3.X docs (confusingly!). The upshot of this long walk is that today's `range` doesn't consume much space, because it produces numbers only on demand.

Although the preceding range results may be useful all by themselves, they tend to come in most handy within for loops. For one thing, they provide a simple way to repeat an action a specific number of times. To print three lines, for example, use a range to generate the appropriate number of integers:

```
python >> for i in range(3):
...     print(i, 'Pythons') ... 0 Pythons 1 Pythons 2 Pythons
```

Note that for loops force results from `range` automatically, so we don't need to use a list wrapper here. In fact, we shouldn't: letting `range` produce its results o

ne at a time uses much less memory than forcing the m all at once.

中文翻译

我们第一个与循环相关的函数 `range` 是一个通用工具，可用于各种场景。我们在第 4 章简单见过它，一路走来也偶尔使用。虽然它常用于在循环中生成索引，但任何需要一串整数的场合你都可以调用它（参见第 11 章的“枚举的名字”技巧，那是一个绝佳例子）。如我们所见，`range` 是一个可迭代（iterable）对象，按需生成项，所以要在 REPL 中一次性显示全部结果，需要把它包在 `list` 调用里。令人惊讶的是，`range` 的结果支持某些序列操作，但不是全部；按第 9 章所述，它被文档重新归类为一种序列对象类型，只不过功能比列表和元组少——而且在“现实基础”上要单薄得多：`python >>> list(range(5)), list(range(2, 5)), list(range(0, 10, 2))` `([0, 1, 2, 3, 4], [2, 3, 4], [0, 2, 4, 6, 8])` `>>> range(5)[2], range(5)[1:3], list(range(5)) + [6, 7]` `(2, range(1, 3), [0, 1, 2, 3, 4, 6, 7])` `>>> range(5) + [6, 7]` `TypeError: unsupported operand type(s) for +: 'range' and 'list'` 撇开归类不谈，`range` 的用法直截了当。一个参数时，`range` 生成从零开始、但不包含该参数值的整数序列。传两个参数时，第一个被视为下界。可选第三个参数给出步长（step）；如果使用它，Python 会把步长依次加到结果的每个整数上（步长默认为 +1）。如果你需要，`range` 也可以是非正的、非递增的：`python >>> list(range(-5, 5))` `[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4]` `>>> list(range(5, -5, -1))` `[5, 4, 3, 2, 1, 0, -1, -2, -3, -4]` 我们将在第 14 章更深入地研究可迭代对象。就这里而言：Python 2.X 曾有一个优化的内

置函数叫 xrange，它类似 range 但不会一次性在内存中构建结果列表；后来在 3.X 中被 range 的生成器行为取代；随后又被 3.X 文档重新包装命名为序列（令人困惑！）。这段漫长旅程的结论是：如今的 range 不占用多少空间，因为它只在被需要时才按需产生数字。尽管前面这些 range 的结果本身就有用，但它们通常在 for 循环中派上最大用场。其中之一：它们提供了一种简单的方式来把某个动作重复特定次数。例如，要打印三行，就用 range 生成相应数量的整数：python >>> for i in range(3): ... print(i,'Pythons') ... 0 Pythons 1 Pythons 2 Pythons 注意 for 循环会自动从 range 中"挤出"结果，所以这里我们不需要 list 包裹。事实上，我们也不应该用：让 range 一次一个地产生结果，比把结果一次性全部强制取出要省内存得多。

代码分析

```
>>> list(range(5)), list(range(2, 5)), list(range(0, 10, 2))
([0, 1, 2, 3, 4], [2, 3, 4], [0, 2, 4, 6, 8])
>>> range(5)[2], range(5)[1:3], list(range(5)) + [6, 7]
(2, range(1, 3), [0, 1, 2, 3, 4, 6, 7])
>>> range(5) + [6, 7]
TypeError: unsupported operand type(s) for +: 'range' and 'list'
```

·解释器如何处理：range 的三个参数是"起始（含）、结束（不含）、步长"。range(5)[2] 走序列索引 → 返回 2；range(5)[1:3] 走切片 → 返回新的 range 对象（不是列表！）；list(range(5)) + [6, 7] 是把 range 转成列表后再拼接 → 合法。但 range(5) + [6, 7] 直接对 range 做 + → 未定义 → TypeError。

·惰性的本质：range 不存储所有数字，只存储"起始、结

束、步长"三个整数 (3.12 里 `sys.getsizeof(range(109))` 只有 48 字节) , 需要时用公式 `start + nstep` 现算。这就是它能处理十亿级范围的原因。

·历史澄清: 2.X 的 `range` 返回真列表 (占内存) , `xrange` 才是惰性的; 3.X 把 `xrange` 改名为 `range` (惰性) , 后来又把它"归类"成序列。三条线合并成今天的一个函数

。

```
>>> for i in range(3):  
...     print(i, 'Pythons')  
...
```

·"重复 N 次"模式: `range(stop)` 给出 `0..stop-1` , 循环体对每个整数运行一次——这就是 Python 版的 `for (int i=0; i<3; i++)` 。但注意: `for` 直接迭代 `range` 对象本身 (不转列表) , 每轮 `next()` 现算出一个整数。

深度理解

·核心概念: `range` 是"虚拟整数序列"——按需生成、几乎零内存。固定次数重复 (`for in range(n)`) 是它最常用的角色。

·底层实现: `range` 对象实现 `len/getitem` (所以支持索引、切片、成员测试) 同时实现 `iter` ; 切片返回新 `range` (保持惰性) 。 + 未实现——因为"虚拟序列拼接"没有意义。它是"鸭子类型"的反例: 像序列但没全实现。

·为什么这样设计: Python 3 的"范围对象化"是一次内存友好的重构——大循环不再先在内存里铺开一个整数列表。`xrange`→`range` 的更名是向 Python 2 程序员传达"这

才是 range 本来的样子"。

·实际问题: `for i in range(n)` (重复 n 次)、`range(len(seq))` (按索引遍历)、`range(a, b, step)` (跳步生成)。它在 `enumerate/zip` 出现前是"要索引必须用它"的唯一途径,如今仍是"重复 N 次"的首选。

·初学者误区: ① `range(5)` 在 REPL 里打印成 `range(0, 5)` 而不是数字列表——以为坏了; ② `range(stop)` 不含 `stop` 值 (`range(5)` 不含 5) ——下标习惯带来的"差一错误"; ③ 在 `for` 里用 `list(range(...))` 白白浪费内存。

·生活例子: `range` 像"电影院的座位号"——你知道起始号、结束号、步长(单号/双号),就能推出任意座位号,不必把全部座位号抄在纸上。

13.5.3 序列扫描: while、range 与 for (Sequence Scans: while, range, and for)

英文原文

The range call is also sometimes used to iterate over a sequence indirectly, though it's often not the best approach in this role. The easiest and fastest way to step through a sequence exhaustively is almost always with a simple for, because Python handles most of the details for you in quick, internal code:

```
python >> X = 'hack'>> for item in X: print(item, end='') # 用 for 自动迭代
```

```
... h a c k
```

Internally, the for loop handles the details of the iteration automatically when used this way. If you really need to take over the indexing logic explicitly (and so sometimes you may), you can do it with a while loop:

```
python >> i = 0 >> while i < len(X): # 用 while 手动迭代
```

```
... print(X[i], end='') ... i += 1 ... h a c k >> i = -1 >> while (i:= i + 1) < len(X): # 手动, 但使用:= 运算符
```

```
... print(X[i], end='') ... h a c k
```

You can also do manual indexing with a for, though, if you use range to generate indexes to iterate through. It's a multistep process—you must ask for the range of the subject's len—but it's sufficient to generate offsets, rather than the items at those offsets:

```
python >> X='hack'>> len(X) # 字符串的长度 4 >> list(range(len(X))) # X 的所有合法偏移 [0, 1, 2, 3] >> >> for i in range(len(X)): print(X[i], end='') # 手动 range/len 迭代
```

```
... h a c k
```

Importantly, because this example is stepping over a list of offsets into X, not the actual items of X, we need to index back into X within the loop to fetch each item. If this seems like overkill, though, it's because it is: there's really no reason to work this hard in this example.

Although the range/len combination is useful in some roles, it's probably not the best option in most. It may run slower, and it's also more code than we need to write. Unless you have a special indexing requirement, you're better off using the simple for loop form in Python:

```
python >> for item in X: print(item, end='') # 能用自动迭代就用
```

As guidelines, use for instead of while whenever possible, and don't use range calls in for loops except as a last resort. This simpler solution is almost always better. Like every good guideline, though, there are plenty of exceptions—as the next section demonstrates.

中文翻译

range 调用有时也用于间接遍历序列，尽管在这个角色上它往往不是最佳方案。彻底遍历序列最简单也最快的方法几乎总是用一个简单 for，因为 Python 在快速的内置代码里为你处理了大部分细节：python >>> X='hack'>>> for item in X: print(item, end='') # 用 for 自动迭代... h a c k 在内部，用这种方式时 for 循环会自动处理迭代的细节。如果你确实需要显式接管索引逻辑（有时候你可能需要），可以用 while 循环：python >>> i = 0 >>> while i < len(X): # 用 while 手动迭代... print(X[i], end='') ... i += 1 ... h a c k >>> i = -1 >>> while (i := i + 1) < len(X): # 手动，但使用 := 运算符... print(X[i], end='') ... h a c k 不过，如果你用 range 生成要遍历的索引，也可以用 for 做手动索引。这是一个多步骤的过程——你必须先问出目标的 l

en 的 range——但它足以生成偏移量，而不是偏移位置上的那些项：python >>> X'hack'>>> len(X) # 字符串的长度4 >>> list(range(len(X))) # X 的所有合法偏移[0, 1, 2, 3] >>> >>> for i in range(len(X)): print(X[i], end='') # 手动 range/len 迭代... h a c k 重要的是，因为本例遍历的是指向 X 的偏移量列表，而不是 X 的实际项，我们必须在循环内部重新索引回 X 才能取出每一项。如果你觉得这小题大做，那确实如此：这个示例里真的没有理由这么费力。尽管 range/len 组合在某些角色中 useful，但在大多数情况下它可能不是最佳选择。它可能运行得更慢，而且比我们需要的代码更多。除非你有特殊的索引需求，否则在 Python 中最好使用简单的 for 循环形式：python >>> for item in X: print(item, end='') # 能用自动迭代就用作为经验法则：尽可能用 for 而不是 while，并且除非万不得已，不要在 for 循环里用 range。这个更简单的方案几乎总是更好。不过，和所有好的经验法则一样，也有大量例外——下一节就会展示。

代码分析

```
>>> X = 'hack'
>>> for item in X: print(item, end=' ') # for
...
h a c k
```

·解释器如何处理：for 直接拿字符串迭代器逐字符产出一——零索引操作、零边界检查（越界由 StopIteration 自然终止）。这是最快、最简洁的扫描方式。

·while 手动版：i < len(X) 里 len(X) 每轮都重新调用（虽然字符串的 len 是 O(1) 缓存）；X[i] 走索引协议。（i：

= i + 1) 海象版先把 i 自增再比较，巧妙省去循环体里的 i += 1。

· range/len 版：range(len(X)) 显式算出 [0,1,2,3]，然后每轮 X[i] 取值。三步变两步，但多了两层间接——作者明说 "true overkill" (确实小题大做)。

· 三条经验法则：① 能用 for 就不用 while；② 能用自动迭代就不用 range/len；③ 有特殊需求（改列表、跳步、并行、要索引）时再升级武器——接下来几节就是这些升级场景。

深度理解

· 核心概念：遍历序列有四种写法由简到繁：自动 for (首选) → while+下标 (要显式控制时) → for + range/len (要偏移时) → for + enumerate (要偏移+值，见 13.5.8)。选哪种由 "是否需要索引" 决定。

· 底层实现：自动 for 走迭代器协议 (C 层快路径)；while+下标每轮做一次 len() 调用、一次下标计算、一次边界隐式检查；range/len 多一层 range 对象的取项——都是 Python 层到 C 层的调用开销差异，量级不大但趋势固定。

· 为什么这样设计：Python 优先 "表达意图"：遍历就说遍历 (自动 for)，不要先说 "我要从 0 到 len-1 的数" 再翻译成下标——后者让读者多动一次脑。

· 实际问题：绝大多数 "读一遍序列" 的任务；需要索引的少数场景 (原地改元素、算法需要位置) 再升级。

·初学者误区：看到 `for i in range(len(seq))` 就复制——先在脑子里问“我真需要 `i` 吗？”；用不到索引时也非要索引，代码变慢且难看。

·生活例子：自动 `for` 像“跟着导游走完全程”；`while`+下标像“自己拿地图一步步定位”；`range/len` 像“先把所有门牌号写出来再挨家找”——同一条街，三种走法，第一种最省心。

13.5.4 序列洗牌器：range 与 len (Sequence Shufflers: range and len)

英文原文

Though not ideal for simple sequence scans, the `range/len` coding pattern used in the prior example does allow us to do more specialized sorts of traversals when required. For example, some algorithms can make use of sequence reordering — to generate alternatives in searches, to test the effect of different value orderings, and so on.

Such cases may require offsets in order to pull sequences apart and put them back together, as in the following; its `range`'s integers provide a repeat count in the first, and a position for slicing in the second:

```
python >> S = 'hack' >> for i in range(len(S)): # 对于重复次数 0..3
```

```
... S = S[1:] + S[:1] # 将前面的项移到末尾... print(S, end='') ... ackh ckha khac hack >> S'hack'>> for i in range(len(S)): # 对于位置 0..3
```

```
... X = S[i:] + S[:i] # 后面部分 + 前面部分... print(X, end='') ... hack ackh ckha khac
```

Trace through these one iteration at a time if they seem confusing. The second creates the same results as the first, though in a different order, and doesn't change the original variable as it goes. Because both slice to obtain parts to concatenate, they also work on any type of sequence, and return sequences of the same type as that being shuffled—if you shuffle a list, you create reordered lists:

```
python >> L = [1, 2, 3, 4] >> for i in range(len(L)):
... X = L[i:] + L[:i] # 适用于任何序列类型... print(X, end='') ... [1, 2, 3, 4] [2, 3, 4, 1] [3, 4, 1, 2] [4, 1, 2, 3]
```

The results of range itself, however, don't make the grade in either coding (they're not true sequences!):

```
python >> L = range(4) >> ...与上一示例相同的代码...
TypeError: unsupported operand type(s) for +: 'range' and 'range'
```

We'll make use of code like this to test functions with different argument orderings in Chapter 18, and will extend it to functions, generators, and more complete permutations in Chapter 20 — it's a widely useful tool.

中文翻译

虽然对简单序列扫描来说不理想，但前一个示例中用到的 `range/len` 编码模式确实让我们能在需要时做更特殊的遍历。例如，有些算法会利用序列的重排（reordering）——在搜索中生成备选方案、测试不同值排列的效果等。这类情况可能需要偏移量，以便把序列拆开再拼回去，就像下面这样：其中 `range` 的整数在第一个中提供重复次数，在第二个中提供切片位置：

```
python >>> S = 'hack' >>> for i in range(len(S)): # 对于重复次数 0..3 ... S = S[1:] + S[:1] # 将前面的项移到末尾... print(S, end='') ... ackh ckha khac
hack >>> S='hack' >>> for i in range(len(S)): # 对于位置 0..3 ... X = S[i:] + S[:i] # 后面部分 + 前面部分... print(X, end='') ... hack ackh ckha khac
```

如果觉得难以理解，请一次一个迭代地跟踪它们。第二个产生了与第一个相同的结果，只是顺序不同，而且它不会在执行过程中改变原始变量。因为两者都通过切片获取要拼接的部分，所以它们也适用于任何类型的序列，并返回与被洗牌的序列同类型的结果——如果你洗牌一个列表，你得到的是重排后的列表：

```
python >>> L = [1, 2, 3, 4] >>> for i in range(len(L)): ... X = L[i:] + L[:i] # 适用于任何序列类型... print(X, end='') .
.. [1, 2, 3, 4] [2, 3, 4, 1] [3, 4, 1, 2] [4, 1, 2, 3]
```

不过，`range` 本身的结果在两种编码中都“不及格”（它们不是真正的序列！）：

```
python >>> L = range(4) >>> ...与上一示例相同的代码...
TypeError: unsupported operand type(s) for +: 'range' and 'range'
```

我们会在第 18 章用类似的代码以不同的参数顺序测试函数，并在第 20 章将它扩展到函数、生成器以及更完整的排列——它是一个广泛有用的工具

代码分析

```
>>> S = 'hack'
>>> for i in range(len(S)): # 0..3
...     S = S[1:] + S[:1] #
...     print(S, end=' ')
...
ackh ckha khac hack
```

·解释器如何处理（旋转循环）： $S = S[1:] + S[:1]$ 每次把当前 S 的第一个字符搬到末尾（循环左移 1 位 4 次）： $hack \rightarrow ackh \rightarrow ckha \rightarrow khac \rightarrow hack$ ，正好输出全部 4 个旋转形态后又回到原样。注意它修改了变量 S （每轮都是新字符串赋值），所以循环结束后 S 又回到 'hack'。

·第二个版本（不动原变量）： $X = S[i:] + S[:i]$ 用固定原串 S 与位置 i 拼出旋转： $i=0$: $hack$ 、 $i=1$: $ackh$ 、 $i=2$: $ckha$ 、 $i=3$: $khac$ ——顺序从“原样”开始，且 S 始终不变。

·类型通用性：对列表 $L[i:] + L[:i]$ 用同样的定位逻辑，得到重排后的列表——切片与拼接保持原类型（对元组得元组）。但若 $L = \text{range}(4)$ ， $L[i:]$ 返回 range 、+ 两个 range 未定义 $\rightarrow \text{TypeError}$ 。这印证了 range “像序列但不全像”。

·为什么这么写：这里是教科书式的“测试函数不同参数顺序”（第 18、20 章将扩展为排列生成）—— range/len 的重复/定位双重角色在此完整展现，正是“有特殊需求时 range 才是对的工具”的注脚。

深度理解

- 核心概念：range/len 模式的两个高级用途——"重复 N 次驱动状态变换"（旋转）、"用位置做重排/分割"。这是它相对自动 for 的真正优势区间。
- 底层实现：每次旋转都新建字符串（字符串不可变）， $O(n)$ 内存分配；若追求性能可改用列表或 collections.deque。切片 $S[i:] + S[:i]$ 的时间复杂度 $O(n)$ 。type 上，切片+拼接是"保类型"操作，而 range 未实现 add。
- 为什么这样设计："能否切片+拼接"成为区分"真序列"与"虚拟序列"（range）的试金石——语言故意不给 range +，因为虚拟对象拼接没有物理意义。
- 实际问题：密码学/密码校验的旋转测试、双端队列轮转、圆桌排序、排列生成（第 20 章）、函数参数顺序全排列测试（第 18 章）。
- 初学者误区：① 忘了第一版会覆盖原变量（S 结尾回到原值只是运气——长度相同旋转一周）；② 拿 range 当列表用去切片拼接，撞上 TypeError 还困惑；③ 以为旋转只能用于字符串——它对任意序列类型都成立。
- 生活例子：旋转循环像"转盘"——每转一下第一位就转到队尾；range/len 里的 i 既是"转了第几圈"也是"当前从哪一刀切开"。

13.5.5 跳过项：range 与切片 (Skipping Items: range and Slices)

英文原文

The prior section showed one valid application for the range/len combination. We might also use this technique to skip items as we go: python >>> S='abcdefghijk'>>> list(range(0, len(S), 2)) [0, 2, 4, 6, 8, 10] >>> for i in range(0, len(S), 2): print(S[i], end='') ... a c e g i k Here, we visit every second item in the string S by stepping over the generated range list. To visit every third item, change the third range argument to be 3, and so on. In effect, using range this way lets you skip items in loops while still retaining the simplicity of the for statement. In many or most cases, though, this is also probably not the "best practice" technique in Python today. If you really mean to skip items in a sequence, the extended three-limit form of the slice expression, presented in Chapter 7, provides a simpler route to the same goal. To visit every second character in S, for example, slice with a stride of 2: python >>> S='abcdefghijk'>>> for c in S[::2]: print(c, end='') ... a c e g i k The result is the same, but substantially easier for you to write and for others to read. The potential advantage to using range here instead is space: slicing makes a copy of the string, while range does not — and hence may save significant memory for very large strings. Naturally, whether your program needs to care depends on what it does.

中文翻译

上一节展示了 range/len 组合的合法应用。我们也可以用这个技巧在遍历时跳过项：python >>> S='abcdefghijk' >>> list(range(0, len(S), 2)) [0, 2, 4, 6, 8, 10] >>> for i in range(0, len(S), 2): print(S[i], end=") ... a c e g i k

这里，我们通过在生成的 range 列表上步进，访问字符串 S 中的每隔一个项。要访问每隔两个项，把第三个 range 参数改成 3，依此类推。实际上，这样使用 range 可以让你在循环中跳过项，同时保留 for 语句的简洁性。不过，在很多甚至大多数情况下，这今天也未必是 Python 中的"最佳实践"技巧。如果你真的想跳过序列中的项，第 7 章介绍的切片表达式 (slice expression) 的扩展三限制形式提供了通往同一目标的更简单路线。例如，要访问 S 中每隔一个的字符，用步长为 2 的切片：python >>> S='abcdefghijk' >>> for c in S[::2]: print(c, end=") ... a c e g i k

结果相同，但写起来和读起来都容易得多。这里改而用 range 的潜在优势是空间：切片会复制字符串，而 range 不会——因此对非常大的字符串可能节省可观的内存。当然，你的程序是否需要在意这一点取决于它在做什么。

代码分析

```
>>> S = 'abcdefghijk'
>>> for i in range(0, len(S), 2): print(S[i], end=' ')
...
a c e g i k
```

·解释器如何处理：range(0, len(S), 2) 生成偏移 [0, 2, 4, 6, 8, 10] (不含超过 len-1 的)；S[i] 逐个取出"隔一个"的字符。缺陷：① 要先算偏移再索引，两步心智；② 无

法一步看到"我在跳步"。

·切片替代：S[::2] 一步到位表达"隔位取"——[start:stop:stride] 三限制形式，stride=2。切片返回一个新字符串（复制），循环再遍历它。几乎总是更可读的选择。

·空间的权衡：S[::2] 对 1 GB 字符串会复制出 500 MB 新对象；range 版零复制。这正是作者的"谨慎"：默认用切片，巨型数据才考虑 range。

·c 与 i 的区别：切片版循环变量是字符本身，range 版是下标再索引——前者更贴近"我要的是字符"这个意图。

深度理解

·核心概念：跳步遍历的两种实现——range 步长（省内存、多步骤）vs 三限制切片（简单、复制）。首选切片，除非数据大到内存敏感。

·底层实现：切片的 stride 在 C 层通过内部偏移计算直接取字节/元素，返回新对象；range 则完全惰性。内存维度上 range 胜，心智维度上切片胜。

·为什么这样设计：Python 鼓励"表达意图"：S[::2] 直接说"每两个取一个"，而 range 版把意图拆成"生成下标 + 取元素"两步。

·实际问题：抽样（每隔 N 个采样）、数据降噪、高维数组步进（NumPy 用同一语法）、分页取行。

·初学者误区：① 每次都写 range(0, len(S), 2) 而忘了 S[::2] 更短；② 在超大列表上滥用切片导致"内存翻倍"仍然懵然不知；③ 切片步长为负时方向反转（S[::-1] 是反转）

——与 range 负数步长的行为一致，但要记住起点终点语义相反。

·生活例子：range 版像"点名时念隔一个的学号再去喊人"，切片版像"直接把名单剪出隔一个的那一行纸"——后者一眼就懂，前者只是在纸特别长的时候才省纸。

13.5.6 修改列表：range 与推导式 (Changing Lists: range and Comprehensions)

英文原文

Another common place where you may use the range /len combination with for is in loops that change a list as it is being traversed. Suppose, for example, that you need to add 1 to every item in a list (maybe you're updating ages at year end). You can try this with a simple for loop, but the result may not be what you want or expect:

```
python >>> L = [10, 20, 30, 40, 50] >>> for x in L: ...  
x += 1 # 改变的是 x, 不是 L! ... >>> L # L 的对象没有变  
[10, 20, 30, 40, 50]
```

```
>>> x # x 不是指向 L 的游标51
```

This doesn't quite work—it changes the loop variable x, not the list L. The reason is somewhat subtle. Each time through the loop, x refers to the next integer already pulled out of the list. In the first iteration, for example, x is integer 10, taken from L. When we then

add to `x` in the loop body with `+=`, it sets `x` to a different object, integer 11, but it does not update the list where 10 originally came from; the new 11 is a piece of memory separate from the list. To really change the list as we march across it, we need to use indexes so we can assign an updated value to each position as we go. The `range/len` combination can produce the required indexes for us:

```
python >>> L = [10, 20, 30, 40, 50] >>> for i in range(len(L)): # 给 L 中的每一项加一... L[i] += 1 # 或者 L[i] = L[i] + 1 ... >>> L [11, 21, 31, 41, 51]
```

When coded this way, the list is changed as we proceed through the loop. There is no way to do the same with a simple `for x in L`, because such a loop iterates through actual items, not their positions. But what about the equivalent while loop? Such a loop requires a bit more work on our part, and might run more slowly depending on your Python, your host device, and perhaps the alignment of planets (you'll see how to check such claims in Chapter 21):

```
python >>> i = 0 >>> while i < len(L): # 与 := 赋值类似... L[i] += 1 ... i += 1 ... >>> L [12, 22, 32, 42, 52]
```

Here again, though, the range solution may not be ideal either. A list comprehension expression of the form:

```
python >>> [x + 1 for x in L] [13, 23, 33, 43, 53]
```

likely runs faster today and would do similar work, albeit without changing the original list in place (we could assign the expression's new list object result back to L, but this would not update any other references to the original list). Because this is such a central looping concept, we'll save a complete exploration of list comprehensions for the next chapter, and tell the rest of the statements story of loops there.

中文翻译

你可能把 range/len 组合与 for 一起使用的另一个常见场合，是在遍历时改变列表的循环中。例如，假设你需要给列表中的每一项加 1（也许你在年末更新年龄）。你可以试着用一个简单 for 循环，但结果可能不是你想要的、也不是你预料的：python >>> L = [10, 20, 30, 40, 50] >>> for x in L: ... x += 1 # 改变的是 x，不是 L! ... >>> L # L 的对象没有变[10, 20, 30, 40, 50] >>> x # x 不是指向 L 的游标51 这不太管用——它改变的是循环变量 x，不是列表 L。原因有点微妙。每次循环，x 都指向刚从列表里取出的下一个整数。例如第一次迭代，x 是取自于 L 的整数 10。当我们在循环体里用 += 给 x 加东西时，它把 x 设为另一个对象——整数 11，但并没有更新 10 原本所在的列表；新的 11 是一块与列表分离的内存。要在遍历过程中真正改变列表，我们需要用索引，这样就能边前进边给每个位置赋更新后的值。range/len 组合正好能为我们生成所需的索引：python >>> L = [10, 20, 30, 40, 50] >>> for i in range(len(L)): # 给 L 中的每一项加一... L[i] += 1 # 或者 L[i] = L[i] + 1 ... >>> L [11, 21, 31, 41, 51] 这样编码，我们

就能在推进循环的过程中改变列表。用简单的 `for x in L` 无法做到同样的事，因为这样的循环遍历的是实际的项，而不是它们的位置。那么等价的 `while` 循环呢？这样的循环需要我们多做一点工作，而且取决于你的 Python、你的设备、也许还有行星对齐的情况，它可能运行得更慢（在第 21 章你会学到如何核验这类断言）：`python >>> i = 0 >>> while i < len(L): # 与 := 赋值类似... L[i] += 1 ... i += 1 ... >>> L [12, 22, 32, 42, 52]` 不过这里也一样，`range` 方案可能也不理想。列表推导（list comprehension）表达式这种形式：`python >>> [x + 1 for x in L] [13, 23, 33, 43, 53]` 今天很可能跑得更快，做的活也差不多——尽管它不会原地修改原列表（我们可以把表达式产生的新列表对象赋回给 `L`，但这不会更新指向原列表的其他引用）。因为这是一个核心的循环概念，我们将在下一章完整探索列表推导，并在那里讲完循环语句故事的其余部分。

代码分析

```
>>> L = [10, 20, 30, 40, 50]
>>> for x in L:
...     x += 1           # x L
...
>>> L                    # L
[10, 20, 30, 40, 50]
```

·解释器如何处理（经典陷阱）：Python 的对象模型是“变量 = 名字绑定到对象”。`for x in L` 每轮把 `x` 绑定到列表里的那个整数对象（如 10）。`x += 1` 对整数而言是“创建新对象 11 并把 `x` 改绑到它”——列表 `L` 中第 0 个槽位仍然保存着原对象 10。循环结束 `x` 停在 51（最后一项 50 + 1）。列表项是对象引用，= 不改引用源。

·正确改法 (range/len) : $L[i] += 1$ 是"取槽位 i 的对象 → 加 1 得新对象 → 写回槽位 i " (等价于 $L[i] = L[i] + 1$)。这时写的是列表的存储位置, 所以列表真的变了。

·推导式版: $[x + 1 \text{ for } x \text{ in } L]$ 生成新列表 (不碰原列表), 通常最快。要"替换"原变量就 $L = [x+1 \text{ for } x \text{ in } L]$ ——但注意: 其他指向旧列表的变量不会跟着更新, 这就是"原地修改 vs 新建"的本质区别。

深度理解

·核心概念: 本小节是全章最重要的"顿悟点"之一: $\text{for } x \text{ in } L$ 中 x 只是"指向当前项对象的名字", 给它赋值不会写回列表。要改列表必须通过索引或列表自己的方法。

·底层实现: 列表是"对象引用数组"。 $L[i]$ 读 (LOADSUBSCR) 取出引用, $L[i] = v$ 写 (STORESUBSCR) 替换该槽位的引用。 $x += 1$ 走 LOADFAST/BINARYOP/STOREFAST——完全绕过节点。整数不可变, 因此 $+1$ 必然产生新对象, 不存在"就地修改整数"。

·为什么这样设计: "一切皆对象引用"的统一模型简化了语言, 代价是程序员要理解"绑定"与"修改"的区别——这是 Python 学习曲线的第一个陡坡。

·实际问题: 批量数据更新 (批量加价、缓存刷新、数值归一化) 是"改列表内容"的典型; 而"生成新列表"的业务 (挖掘、转换、采样) 用推导式更好。

·初学者误区: ① $\text{for } x \text{ in } L: x += 1$ 后惊讶列表没变 (本书正面警告); ② 用 $\text{for } x \text{ in } L:$ 中替换 x 来"改"列表——替换永远是无效的; ③ 以为推导式会原地修改。

·生活例子：列表像"抽屉柜"， x 像"你正拿在手里看的東西"
——看了 10 就把它改成 11，是把心的 11 放回抽屉嗎？
不，抽屉里的 10 还在原地。要真正換成 11，必須"打開第 i 個抽屜，把 11 放進去" ($L[i] = \dots$)。

13.5.7 并行遍历: zip (Parallel Traversals: zip)

英文原文

Our next loop coding technique adds to its bag of tricks. As we've seen, the range built-in allows us to traverse sequences with for in a nonexhaustive fashion. In a similar spirit, the built-in zip function allows us to use for loops to visit multiple sequences in parallel — not overlapping in time, but during the same loop. In basic operation, zip takes one or more arguments (sequences or other iterables) and returns a series of tuples that pair up parallel items taken from those arguments. For example, suppose we're working with two lists of data paired by position:

```
python >>> L1 = [1, 2, 3, 4] >>> L2 = [5, 6, 7, 8] To combine the items in these lists, we can use zip to create a list of tuple pairs. Like range, zip is an iterable object, so we must wrap it in a list call to collect and display all its results at once (again, the next chapter will be more formal about iterables like this):
```

```
python >>> zip(L1, L2) # 一个生成配对的 iterable (可迭代对象) <zip object at 0x026523C8>
```

```
>>> list(zip(L1, L2)) # 需要 list() 才能看到所有结果[(1, 5), (2, 6), (3, 7), (4, 8)]
```

Such a result may be useful in other contexts as well, but when wedded with the for loop, it supports parallel iterations:

```
python >>> for (x, y) in zip(L1, L2): ... print(f'{x} + {y} => {x + y}') ... 1 + 5 => 6 2 + 6 => 8 3 + 7 => 10 4 + 8 => 12
```

Here, we step over the result of the zip call — that is, the pairs of items pulled from the two lists. Notice that this for loop again uses the tuple (a.k.a. sequence) assignment form we met earlier to unpack each tuple in the zip result. The first time through, it's as though we ran the assignment statement $(x, y) = (1, 5)$; and so on. The net effect is that we scan both L1 and L2 in our loop. To be sure, we could achieve a similar effect with a while loop that handles indexing manually — like the following that produces the same output as the preceding:

```
python >>> i = -1 >>> while (i := i + 1) < len(L1): ... print(f'{L1[i]} + {L2[i]} => {L1[i] + L2[i]}')
```

But this requires noticeably more code, and hence would likely run slower than the for/zip approach. Moreover, it's no better on space: being an iterable, zip makes just one pair per loop, and so does not consume memory needlessly. The clincher, though, is that this is not really equivalent to zip — for reasons disclosed in the next

中文翻译

我们下一个循环编码技巧又为工具袋增添了一招。如我们所见，内置 `range` 让我们能用 `for` 以非穷尽（nonexhaustive）的方式遍历序列。与之精神相同，内置 `zip` 函数允许我们用 `for` 循环并行（in parallel）访问多个序列——不是时间上重叠，而是在同一个循环期间。基本操作上，`zip` 接受一个或多个参数（序列或其他可迭代对象），返回一系列元组，把来自这些参数的并行项配对起来。例如，假设我们正在处理两个按位置配对的数据列表：

```
python >>> L1 = [1, 2, 3, 4]
>>> L2 = [5, 6, 7, 8]
```

要组合这些列表中的项，我们可以用 `zip` 创建一个元组对的列表。和 `range` 一样，`zip` 也是一个可迭代对象，所以必须用 `list` 调用包裹，才能一次性收集并显示它全部的结果（再次强调，下一章会正式讲解这类可迭代对象）：

```
python >>> zip(L1, L2) # 一个生成配对的 iterable（可迭代对象）
<zip object at 0x026523C8>
>>> list(zip(L1, L2)) # 需要 list() 才能看到所有结果
[(1, 5), (2, 6), (3, 7), (4, 8)]
```

这样的结果在其他场景中也可能有用，但当它与 `for` 循环结合时，就支撑起了并行迭代：

```
python >>> for (x, y) in zip(L1, L2): ... print(f'{x} + {y} => {x + y}') ...
```

1 + 5 => 6 2 + 6 => 8 3 + 7 => 10 4 + 8 => 12

这里，我们遍历 `zip` 调用的结果——也就是从两个列表取出的配对项。注意这个 `for` 循环再次使用了我们之前见过的元组（即序列）赋值形式，来解包 `zip` 结果中的每个元组。第一次循环就像执行了赋值语句 `(x, y) = (1, 5)`；依此类推。净效果是我们在一个循环中同时扫描了 `L1` 和 `L2`。当然，我们也可以用处理手动索引的 `while` 循环达到类

似效果——比如下面这段产生与上述相同输出的代码：python >>> i = -1 >>> while (i := i + 1) < len(L1): ... print(f'{L1[i]} + {L2[i]} => {L1[i] + L2[i]}') 但这明显需要更多代码，因而很可能比 for/zip 方案运行得慢。而且它在空间上并无优势：作为可迭代对象，zip 每轮只生成一对，不会无谓地消耗内存。不过，最致命的是：它其实并不等同于 zip——原因在下一节揭晓。

代码分析

```
>>> for (x, y) in zip(L1, L2):  
...     print(f'{x} + {y} => {x + y}')
```

· 解释器如何处理：zip(L1, L2) 返回一个 zip 迭代器。每轮 for 调用它的 next：zip 内部同时对 L1 和 L2 各自的迭代器调用 next()，把得到的两项组成一个二元元组返回——注意两个 next() 是“背靠背”调用的，zip 对象本身不存储任何元素。(x, y) 再解包。

· zip 是惰性的：zip(L1, L2) 直接打印显示的是 <zip object...> (对象地址)，必须 list() 才能看到 [(1,5),...]——它和 range 一样不预先生成全部结果，for 循环边遍历边取。

· while 手写版的缺陷：① 先 $i := i + 1$ 再比较，海象自增逻辑隐蔽；② $L1[i]/L2[i]$ 两次索引，还要自己保证 len(L1) 不越界——若 L2 更短则 IndexError！这正是作者说的“并不真正等同于 zip”的核心：zip 自动以最短序列为准截断，手写版完全不管长度差异。

深度理解

·核心概念：zip 把"多个序列的第 k 项"捆绑成第 k 个元组，实现"同一索引下并行遍历"。它是惰性迭代器，只按需产出一对。

·底层实现：zip 迭代器的 next 会调用每个输入迭代器的 next()；任一输入耗尽了，就抛 StopIteration 结束整个 zip——因此结果长度 = 最短输入长度（这就是"截断"的机制来源）。

·为什么这样设计：数据配对（学号/姓名、键/值、时间/测量）是永恒的需求。手写 while + 双索引极易出越界 bug；zip 把"成对取"打包成原语，长度不齐时裁剪而非报错，符合"大多数时候取公共部分"的实际意图。

·实际问题：并行列表求和、乱序字典构造、表列配对、机器学习特征向量拼接——凡"两个序列等长对应"的场景全是 zip 的主场。

·初学者误区：① 忘了 .zip 是惰性对象，REPL 里直接打印只有地址；② 期望 zip 输出比最短序列长——不会，短者为界；③ 若想要"补齐"而非截断（zip_longest），Python 标准库 itertools 提供，本书第 20 章也会提到垫齐思路。

·生活例子：zip 像"两条传送带并排"——左边出一件、右边出一件，机械臂同时抓俩；其中一条没货了，整条线就停。

13.5.7.1 更多 zip: 大小与截断 (More on zip: size and truncation)

英文原文

For the record, the `zip` function is more general than the prior example suggests. For instance, it both is an iterable and accepts any type of iterable object, including range results, input files, and more:

```
python >>> list(zip(range(4), 'hack')) [(0, 'h'), (1, 'a'), (2, 'c'), (3, 'k')]
```

In addition, `zip` is not just for two-item pairs: it accepts any number of arguments, of any size. The following, for example, builds a list of three-item tuples for three arguments, with items from each sequence — essentially projecting by columns:

```
python >>> T1, T2, T3 = (1, 2, 3), (4, 5, 6), (7, 8, 9) >>> > T3 (7, 8, 9)
>>> list(zip(T1, T2, T3)) # 3 个参数各 3 个值 => 3 个 3 项元组[(1, 4, 7), (2, 5, 8), (3, 6, 9)]
```

And formally speaking, for N arguments that contain M items, `zip` gives us an M -long series of N -ary tuples:

```
python >>> list(zip(T1, T2)) # 2 个参数各 3 个值 => 3 个 2 项元组[(1, 4), (2, 5), (3, 6)]
```

When argument lengths differ, `zip` truncates the series.

es of result tuples at the length of the shortest sequence. To demo, the following zips two strings to pick out characters in parallel, but the result has only as many tuples as the length of the shortest sequence (formally again, M in the prior definition is really the minimum of arguments' lengths):

```
python >>> S1 = 'abc'>>> S2 = 'xyz123'>>> >>> list
(zip(S1, S2)) # 在最短序列长度处截断[('a','x'), ('b','y'), ('c',
', 'z')]
```

To pad instead of truncating, you can write loop code to pad results yourself — as we will in Chapter 20, after we've had a chance to study some additional iteration concepts that make it a fair fight.

中文翻译

郑重说明一下，zip 函数比前面示例所暗示的更通用。例如，它既是一个可迭代对象，又接受任何类型的可迭代对象作为参数，包括 range 的结果、输入文件，以及更多：python >>> list(zip(range(4), 'hack')) [(0, 'h'), (1, 'a'), (2, 'c'), (3, 'k')] 此外，zip 不仅做两项配对：它接受任意数量的参数，任意大小。例如，下面为三个参数构建一个三元元组列表，元素来自每个序列——本质上是按列投影：python >>> T1, T2, T3 = (1, 2, 3), (4, 5, 6), (7, 8, 9) >>> T3 (7, 8, 9) >>> list(zip(T1, T2, T3)) # 3 个参数各 3 个值 => 3 个 3 项元组[(1, 4, 7), (2, 5, 8), (3, 6, 9)] 正式地说，对有 M 个项的 N 个参数，zip 给我们一个长度为 M 的 N 元元组序列：python >>> list(zip(T1, T2)) # 2 个参数各 3 个值 => 3 个 2 项元组[(1, 4), (2, 5), (3, 6)] 当参数长度不一时

，zip 在最短序列的长度处截断 (truncate) 结果元组序列。做个演示：下面的代码并行打包两个字符串以挑出字符，但结果元组数量只有最短序列的长度（再次正式地说，上面定义中的 M 实际上就是参数长度的最小值）：`python >>> S1 = 'abc'>>> S2 = 'xyz123'>>> >>> list(zip(S1, S2))` # 在最短序列长度处截断`[('a','x'), ('b','y'), ('c','z')]` 要"补齐 (pad)"而不是截断，你可以自己写循环代码来补足结果——就像我们在第 20 章要做的，等我们有机会研究一些额外的迭代概念能与之公平对决之后。

代码分析

```
>>> list(zip(range(4), 'hack'))  
[(0, 'h'), (1, 'a'), (2, 'c'), (3, 'k')]
```

· 解释器如何处理：range 迭代器产出 0,1,2,3，字符串迭代器产出'h','a','c','k'——zip 按位置配对，于是 [(0,'h'), ..]。类型混合（整数+字符）完全没问题，zip 不关心元素类型，只关心"配对"。

· N 参投影：zip(T1, T2, T3) 第一轮从三个元组各取第一个元素 → (1, 4, 7)，即"按列取"。这等价于把三行数据转置成三列——zip(matrix)（解包参数）是矩阵转置的经典写法。

· 截断语义：'abc' 长 3、'xyz123' 长 6 → 结果只有 3 对。zip 的结束条件是"任一输入耗尽"——这是它与手写 while 版（越界爆炸）的本质区别。

· 补齐预告：要"较短者摊开成 None"，标准库 `itertools.zip_longest(..., fillvalue=None)` 是现成工具；本书第 20 章会自己手写补全循环以深化理解。

深度理解

·核心概念：zip 的三个通用属性：任意参数个数、任意类型可迭代对象、以最短为准截断。这三个属性共同支撑起"转置""投影""并行扫描"等高级用法。

·底层实现：zip 的 next 循环调用每个输入迭代器的 next(), 遇到第一次 StopIteration 就结束——"最短截断"是这条实现的自然结果，而非刻意裁剪。

·为什么这样设计：截断体现了"公共长度"直觉——等长时完美，长度不齐时宁缺毋滥。Python 选择"安全裁剪"而非"抛异常"，避免在常见场景（两个长度几乎一致的序列）频繁出错。

·实际问题：矩阵转置、表格行列互换、键值配对构造字典（见下节）、把三列日志合成三元组记录。

·初学者误区：① 期待 zip 元素多于最短序列；② 用 zip 处理超长输入还转成 list——zip 惰性但 list() 会一口气全算；③ 忘了 zip(seq) 是转置（解包后按"列"取），语义与直接的 zip(a, b) 不同。

·生活例子：N 参 zip 像"合唱对位"——每部（声部）各唱一个音，同一小节三个声部合起来就是三音和弦；有部先没声了（耗尽），整小节就结束。

13.5.7.2 zip 的更多角色：字典 (More zip roles: dictionaries)

英文原文

Fine points aside, parallel traversals with `zip` are also useful in dictionary construction. We met this technique in Chapter 8, but here's a quick refresher in the context of looping statements. As we learned earlier, you can always create a dictionary by calling `dict`, assigning to keys over time, or coding a dictionary literal like the following:

```
python >>> D1 = {'app': 1, 'script': 3, 'program': 5} #  
或者 dict(key=value, ...) >>> D1 {'app': 1, 'script': 3, 'program': 5}
```

What to do, though, if your program obtains dictionary keys and values at runtime, after you've coded your script? For example, the following may be collected from a user, a file, or any other dynamic source:

```
python >>> keys = ['app', 'script', 'program'] >>> vals  
= [1, 3, 5] One way to turn these into a dictionary is to  
zip the lists and step through them in parallel with  
a for loop:
```

```
python >>> list(zip(keys, vals)) [('app', 1), ('script', 3),  
('program', 5)]
```

```
>>> D2 = {} >>> for (k, v) in zip(keys, vals): D2[k] = v  
... >>> D2 {'app': 1, 'script': 3, 'program': 5}
```

As suggested earlier in this book, though, you can skip the for loop altogether in this context, and simply pass the zipped keys/values lists to the built-in `dict` constructor:

onstructor call:

```
python >>> D3 = dict(zip(keys, vals)) >>> D3 {'app': 1, 'script': 3, 'program': 5}
```

The built-in name `dict` is really a type name (you'll learn about type names, and subclassing them, in Chapter 32). Calls to it are object construction requests, but also perform a to-dictionary conversion here. In the next chapter, you'll also learn more about related but richer concepts — list comprehensions, which build lists in expressions, and their dictionary comprehensions kin, which are an alternative to both for statements and `dict` for zipped key/value pairs:

```
python >>> {k: v for (k, v) in zip(keys, vals)} {'app': 1, 'script': 3, 'program': 5}
```

中文翻译

细节撇开不谈，用 `zip` 并行遍历在字典构造中也很有用。我们在第 8 章见过这一技巧，但这里在循环语句的语境下快速复习一下。如前面所学，你总可以通过调用 `dict`、随时间逐一给键赋值，或编写如下所示的字典字面量来创建字典：

```
python >>> D1 = {'app': 1, 'script': 3, 'program': 5} #  
或者 dict(key=value, ...) >>> D1 {'app': 1, 'script': 3, 'program': 5} 不过，如果你的程序在你写完脚本之后才在运行时获得字典的键和值呢？例如，下面的键和值可能是从用户、文件或其他任何动态来源收集来的：python >>> keys = ['app', 'script', 'program'] >>> vals = [1, 3, 5] 把它
```


们变成字典的方法之一是用 `zip` 把两个列表打包，再用 `for` 循环并行遍历：`python >>> list(zip(keys, vals))` `[('app', 1), ('script', 3), ('program', 5)]` `>>> D2 = {}` `>>> for (k, v) in zip(keys, vals): D2[k] = v ... >>> D2` `{'app': 1, 'script': 3, 'program': 5}` 不过，如本书前面建议的，这种场景下你可以完全跳过 `for` 循环，直接把打包后的键/值列表传给内置 `dict` 构造调用：`python >>> D3 = dict(zip(keys, vals))` `>>> D3` `{'app': 1, 'script': 3, 'program': 5}` 内置名字 `dict` 实际上是一个类型（`type`）名（你会在第 32 章学习类型名字以及如何子类化它们）。调用它就是对象构造请求，但在这里也执行了一次到字典的转换。在下一章你还会学到相关但更丰富的概念——在表达式中构建列表的列表推导，以及它们的表亲字典推导（`dictionary comprehension`），对“打包的键/值对”而言，它们既可以替代 `for` 语句，也可以替代 `dict`：`python >>> {k: v for (k, v) in zip(keys, vals)}` `{'app': 1, 'script': 3, 'program': 5}`

代码分析

```
>>> D2 = {}
>>> for (k, v) in zip(keys, vals): D2[k] = v
...
>>> D3 = dict(zip(keys, vals))
>>> {k: v for (k, v) in zip(keys, vals)}
```

· 三种构造方式对比：① `for` 循环版本显式、渐进（可加验证/处理逻辑）；② `dict(zip(keys, vals))` 是最简洁的惯用法——`zip` 迭代器被 `dict` 构造器直接“嚼”掉，`dict` 接受（`key, value`）对的可迭代对象；③ 字典推导 `{k: v for ...}` 是功能更全的表达式（可加过滤 `if`），是“让 Python 干活”的极致。

·解释器如何处理：dict(zip(...)) 的底层仍是循环——dict 的内部实现遍历传入的可迭代对象，把每个二元组当作键值对插入哈希表。你省的是"手写循环"的样板，不是循环本身。

·类型转换视角：dict() 是"可迭代的键值对 → 字典"的转换器；这也是"构造调用 = 转换"的普遍模式（list()、tuple()、set() 同理）。

深度理解

·核心概念：zip + dict/字典推导 = "两个列表合并成字典"的三级进化：for 循环 → dict() 构造器 → 推导式。等级越高越表达"意图"。

·底层实现：字典推导与列表推导共享"C 级循环派生子类"的实现（PEP 709 之前的 createframe 在内层，3.12 后仍是独立迭代代码路径），它们比等价的 Python for 循环更快，因为迭代在 C 层而非字节码解释层完成。

·为什么这样设计："键值对列表 → 字典"是数据入库、配置解析、CSV 转字典的刚需，所以语言给了三个级别的工具，程序员按"需要多少控制"来选。

·实际问题：从数据库/表格读出的两列合成映射、配置键值对加载、把响应头列表转 dict、序列化前的字典重排。

·初学者误区：① 只知 dict(key=value) 静态写法，遇到运行时数据就束手无策；② 键重复时后值覆盖前值（zip 拼接的天然结果，需知晓）；③ 忘了键必须可哈希——列表不能当键。

·生活例子：像"两条名单对齐成一张表"——学号列 keys、姓名列 vals，dict 构造器就是"把两列钉在一起成一张联系表"的装订机。

13.5.8 偏移与项：enumerate (Offsets and Items : enumerate)

英文原文

Our final loop-helper function is designed to support dual usage modes. Earlier, we discussed using range to generate the offsets of items in a string, rather than the items at those offsets. In some programs, though, we need both: the item to use, plus an offset as we go. This might be coded with a for loop that also keeps a counter of the current offset:

```
python >> S='hack'>> offset = 0 >> for item in S:
... print(item,'appears at offset', offset) ... offset += 1
.. h appears at offset 0 a appears at offset 1 c appears
at offset 2 k appears at offset 3
```

This works, but Python has a built-in function named enumerate that does the job for us — its net effect is to give loops a counter "for free," without sacrificing the simplicity of automatic iteration:

```
python >> S='hack'>> for (offset, item) in enumerate(S):
```

```
... print(item,'appears at offset', offset) ... h appears at  
offset 0 a appears at offset 1 c appears at offset 2 k a  
ppears at offset 3
```

As for range and zip, the enumerate function's result is an iterable — a kind of object that supports the iteration protocol that we will dive into in the next chapter. In short, it has a method called by the next built-in function, which returns an (index, value) tuple each time through the loop. The for steps through these tuples automatically, which allows us to unpack their values with tuple assignment, much as we did for zip:

```
python >> E = enumerate(S) >> E <enumerate object  
at 0x10ebd7880> >> next(E) (0,'h') >> next(E) (1,'a')  
>> next(E) (2,'c')
```

We don't normally see this machinery because all iteration contexts — including list comprehensions, the main subject of Chapter 14 — run the iteration protocol automatically:

```
python >> [c i for (i, c) in enumerate(S)] ['','a','cc','kkk']  
>> for (ix, line) in enumerate(open('data.txt')):
```

```
... print(f'{ix}) {line.rstrip()}') ... 0) Testing file IO 1) Learning Python, 6E 2) Python 3.12
```

To fully understand iteration concepts like enumerate and list comprehensions, though, we need to move on to the next chapter for a deeper dissection.

中文翻译

我们最后一个循环辅助函数被设计成支持双重用法模式。前面我们讨论过用 `range` 生成字符串中项的偏移量，而不是偏移量上的那些项。但在某些程序中，我们两者都要：要用的项，加上遍历时的偏移量。这可以用一个同时维护当前偏移计数器的 `for` 循环来编码：python >>> S = 'hack'>>> offset = 0>>> for item in S: ... print(item,'appears at offset', offset) ... offset += 1 ... h appears at offset 0 a appears at offset 1 c appears at offset 2 k appears at offset 3 这能工作，但 Python 有个名为 `enumerate` 的内置函数替我们干这活——它的净效果是为循环"免费"提供一个计数器，同时不牺牲自动迭代的简洁性：python >>> S = 'hack'>>> for (offset, item) in enumerate(S): ... print (item,'appears at offset', offset) ... h appears at offset 0 a appears at offset 1 c appears at offset 2 k appears at offset 3 与 `range` 和 `zip` 一样，`enumerate` 函数的结果是一个可迭代对象——一种支持迭代协议的对象，我们将在下一章深入探讨。简而言之，它有一个由内置 `next` 函数调用的方法，每次循环返回一个 (index, value) (索引, 值) 元组。`for` 自动遍历这些元组，让我们能用元组赋值解包它们的值，就像我们为 `zip` 做的那样：python >>> E = enumerate(S)>>> E <enumerate object at 0x10ebd7880>>> next(E) (0,'h')>>> next(E) (1,'a')>>> next(E) (2,'c') 我们通常看不到这个机制，因为所有迭代上下文——包括列表推导 (第 14 章的主角) ——都会自动运行迭代协议：python >>> [c i for (i, c) in enumerate(S)] ['','a','cc','kkk']>>> for (ix, line) in enumerate(open('dat

a.txt')): ... print(f'{ix}) {line.rstrip()}') ... 0) Testing file IO

1) Learning Python, 6E 2) Python 3.12 不过，要充分理解 enumerate 和列表推导这类迭代概念，我们需要进入下一章做更深入的剖析。

代码分析

```
>>> E = enumerate(S)
>>> E
<enumerate object at 0x10ebd7880>
>>> next(E)
(0, 'h')
>>> next(E)
(1, 'a')
>>> next(E)
(2, 'c')
```

·解释器如何处理（手工驱动迭代器）：enumerate(S) 创建 enumerate 迭代器。手动调用内置 next(E)（对应底层 next 方法）会推进它：内部维护一个只增计数器 count（初始为 0），每次 next() 先从底层迭代器（字符串 S 的迭代器）取下一个字符，再返回 (count, char) 并把 count 加 1。这就是整个迭代协议在“手工视角”下的样子——下一章会正式定义 iter/next。

·与手动计数器版对比：offset = 0 + offset += 1 每轮都要两行；enumerate 把它们折叠为“取 (i, item)”——少了一个极易忘更新的变量（忘了 offset += 1 就全输出同一个 0）。

·推导式中的应用：[c i for (i, c) in enumerate(S)] ——推导式的迭代器部分 for (i, c) in enumerate(S) 与普通 for 完全同构，只是同时完成“取下标+取字符+解包”。'h'0

=', 'a'1='a', 'c'2='cc', 'k'3='kkk'.

深度理解

·核心概念：enumerate = "自动编号的迭代器"。它解决的是"自动迭代（舒服）和要索引（需要）"二选一的矛盾——两者我都要。

·底层实现：enumerate 迭代器持有两个指针：底层迭代器 + 计数器。它的 next 调底层 next()，得到值 v，返回 (counter, v)，计数器 +1。全部在 C 层完成，所以比 Python 手动 offset += 1 快。enumerate(it, start=n) 还支持从任意值起编。

·为什么这样设计："处理数据时知道这是第几行/第几条/第几个"是打印日志块号、报表行号、调试定位的刚需。与其让每个程序员各自维护计数器（漏更就错），不如把"计数"直接做进迭代器官。

·实际问题：逐行读取文件并给行号 (for ix, line in enumerate(f))、给搜索结果编号、调试打印每次循环的迭代序号、带序号的列表推导。

·初学者误区：① 拿 range(len(seq)) 再手动 seq[i] 绕远路——enumerate 一步到位还要更"符合语义"；② 忘了解包 (for i, item in enumerate(...), 写一个目标变量会收到元组)；③ 想从 1 开始编号时手写 i+1——用 enumerate(x, start=1) 更清晰。

·生活例子：enumerate 像"发号牌"——你来一个人（项），我就给你一个当前号码牌（index），号码从 0 起逐次递增；开演唱会入场（读文件行）时大家都在用这个发

号机器，不必自己手画号码。

13.6 章节小结 (Chapter Summary)

英文原文

In this chapter, we explored Python's looping statements and their related tools. We looked at the while and for loop statements in depth, and we learned about their associated else clauses. We also studied the break and continue statements, which have meaning only inside loops, and met several built-ins commonly used in for loops, including range, zip, and enumerate, although some of the details regarding their roles as iterables were intentionally cut short. In the next chapter, we continue the iteration story by discussing list comprehensions and the iteration protocol in Python — concepts strongly related to for loops. There, we'll also fill in the rest of the picture behind the iterable tools we met here, such as range and zip, and study some of the subtleties of their operation. As always, though, before moving on let's exercise the knowledge you've picked up here with a quiz.

中文翻译

本章我们探索了 Python 的循环语句及其相关工具。我们深入研究了 while 和 for 循环语句，并学习了它们各自的 else 子句。我们还学习了只在循环内部有意义的 break 和 co

ntinue 语句，并认识了几个常用于 for 循环的内置函数，包括 range、zip 和 enumerate——尽管关于它们作为可迭代对象角色的部分细节被刻意简短带过。在下一章，我们将通过讨论 Python 中的列表推导和迭代协议来继续迭代的故事——这些概念与 for 循环密切相关。在那里，我们还将补全我们在这里遇到的可迭代工具（如 range 和 zip）背后的其余图景，并研究它们操作中的一些微妙之处。不过，和往常一样，在继续之前，让我们用一个小测验来练练你在这里学到的东西。

深度理解

·核心概念：本章是“循环语法”的终点、“迭代协议”的起点——作者刻意按下暂停键，把 range/zip/enumerate 的“可迭代对象本性”铺垫到下一章深入。

·全章知识地图：while/for 两种循环骨架 → break/continue/pass/else 四个控制词 → range（索引/重复）→ zip（并行/字典）→ enumerate（索引值兼得）→ 文件扫描器应用（本章末）。第 14 章会把“为什么 for 能遍历一切”这个底层问题彻底解开。

·为什么在这里“留白”：List comprehension、迭代协议、生成器都是难点，一次性全讲会信息过载。Lutz 的教学节奏是“先会用，再懂原理”。

13.7 测试你的知识：测验 (Test Your Knowledge: Quiz)

英文原文

1. What are the main functional differences between while and for loops? 2. What's the difference between break and continue? 3. When is a loop's else clause executed? 4. How can you code a counter-based loop in Python? 5. What can a range be used for in a for loop?

中文翻译

1. while 和 for 循环之间主要的functional差异是什么? 2. break 和 continue 的区别是什么? 3. 循环的 else 子句在什么时候执行? 4. 在 Python 中如何编写基于计数器的循环? 5. range 在 for 循环中可以用于做什么?

深度理解

·测验的意义：这五题覆盖全章五个支柱——循环选型、循环控制、else 语义、计数方式、range 的应用面。能不看答案答全，说明本章核心已内化。

·自测思路：每题都对应一个“易错点”：for/while 按用途选型、break/continue 作用位置不同、else 只在“未 break 的正常退出”执行、计数优先 for+range 而非 while、range 的四大角色（重复/索引/跳步/改列表）且多可被替代。

13.8 测试你的知识：答案 (Test Your Knowledge: Answers)

英文原文

1. The while loop is a general looping statement, but the for is designed to automatically iterate across items in a sequence or other iterable. Although the while can imitate the for with counter loops, it takes more code and might run slower. 2. The break statement exits a loop immediately (control flow winds up below the entire while or for loop statement), and continue jumps back to the top of the loop (control flow winds up positioned just before the test in while or the next item fetch in for). 3. The else clause in a while or for loop will be run once as the loop is exiting, if and only if the loop exits normally (i.e., by a false test in while or an empty object in for), without running into a break statement. A break exits the loop immediately, skipping the else part on the way out (if there is one). 4. Counter loops can be coded with a while statement that keeps track of the index manually, or with a for loop that uses the range built-in function to generate successive integer offsets. Neither is the preferred way to code in Python, if you need to simply step across all the items in a sequence. Instead, use a simple for loop without range or counters, whenever possible; it will be easier to code and usually quicker to

o run. 5. The range built-in can be used in a for loop to implement a fixed number of repetitions, to scan by offsets instead of items at offsets, to skip successive items as you go, and to change a list while stepping across it. None of these roles requires range, and most have alternatives — scanning actual items, three-limit slices, and list comprehensions are often better solutions today (despite the natural inclinations of ex-C programmers to want to count things!).

中文翻译

1. while 循环是通用的循环语句，而 for 被设计来自动遍历序列或其他可迭代对象中的项。虽然 while 可以用计数器循环模仿 for，但它需要更多代码，而且可能运行得更慢。
2. break 语句会立即退出循环（控制流最终落在整个 while 或 for 循环语句的下面），而 continue 跳回循环顶部（控制流最终落在 while 的测试之前，或 for 的下一次取项之前）。
3. 在 while 或 for 循环中，else 子句会在循环退出时运行一次，当且仅当循环正常退出（即在 while 中测试为假，或在 for 中对象为空）且没有遇到 break 语句时。break 会立即退出循环，并在离开时跳过 else 部分（如果有的话）。
4. 计数器循环可以用手动维护索引的 while 语句编码，也可以用使用内置 range 函数生成连续整数偏移的 for 循环编码。如果只是要简单遍历序列中的所有项，这两种都不是 Python 中的首选写法。相反，尽可能用一个不含 range 或计数器的简单 for 循环；它编码更容易，运行通常也更快。
5. 内置 range 可以在 for 循环中用于：实现固定次数的重复、按偏移而不是偏移上的项来扫描、在遍历时跳

过连续的项，以及在遍历时修改列表。这些角色没有一个必须用 `range`，大多数都有替代方案——遍历实际项、三限制切片和列表推导今天往往是更好的方案（尽管 `ex-C` 程序员有数数的天然冲动！）。

深度理解

·核心启示：官方答案的潜台词是"简单 `for` 优先"——`range`/计数只有特殊需求才用。这正是本章反复强调的"抵抗计数的诱惑"的总结陈词。

·易错点再强调：`else` 的条件"正常退出" = 测试变假 (`while`) / 项取空 (`for`)，与"循环没跑过"也兼容——三个判据合一。

·下章预告：答案里出现的"三限制切片、列表推导"都指向下一章——推导式是 `for` 循环的"表达式的亲戚"，将部分取代手工循环。

13.9 为什么你会在意：文件扫描器 (WHY YOU WILL CARE: FILE SCANNERS)

英文原文

Loops come in handy anywhere you need to repeat an operation or process something more than once. Because text files contain multiple characters and lines, they are a typical role for loops. Assuming a file's contents can fit in memory, you can load it all at once with the file object's `read` method of Chapter 9: `pyth`

on file = open('data.txt') # 一次性把内容读入字符串
print(file.read()) For more granular access, you can scan by characters instead with either of the following — the second of which doesn't load the whole file and has grown terser with the := named assignment of Chapter 11: python for char in open('data.txt').read(): print(char, end='') file = open('data.txt') while char := file.read(1): # 按字符读取, 空表示文件结尾 print(char, end='') # 每个字符后不要加 \n To read by lines or blocks instead, you can use while loops like the following; binary data is often read by blocks using binary file mode 'rb', but text should use text mode to avoid splitting character bytes: python file = open('data.txt') while line := file.readline(): # 逐行读取 print(line.rstrip()) # 行已经有 \n 换行符 file = open('data.txt') while chunk := file.read(10): # 逐块读取: 最多 10 个字符 print(chunk, end='') # 保留但不添加换行符 To read text files by lines, though, the for loop tends to be easiest to code and may be quickest to run: python for line in open('data.txt').readlines(): print(line.rstrip()) for line in open('data.txt'): # 使用迭代器: 文本输入的最佳选择 (也许) print(line.rstrip()) The first version here uses the file readlines method to load a file all at once into a line-string list, but the second example relies on file iterators to automatically read one line on each loop iteration. The second example is also generally best for text files — besides its simplicity, it works for arbitrarily large files because it doesn't load the entire file into memory

y all at once. The iterator version may also be the quickest, though speed can vary per Python release (we'll study ways to time code later in this book). File read lines calls can still be useful, though — to reverse a file's lines, for example, assuming its content can fit in memory. The reversed built-in works on sequences, but does not accept iterables that generate values; sorted, by contrast, does, so it can order all the lines in a file without loading it in full: `python for line in reversed(open('data.txt').readlines()): print(line.rstrip())` for `line in sorted(open('data.txt')): print(line.rstrip())` See Python's documentation for more on the file-object calls used here — as well as its coverage of tools like `os.popen` that returns a file object connected to a shell command's output (by default), and hence supports the same sort of loops. There's also an `os.popen` example in Chapter 21, and more on the distinctions of text and binary files in Chapter 37 when we dive into Unicode more deeply.

中文翻译

任何需要重复操作或多次处理某个东西的地方，循环都派得上用场。因为文本文件包含多个字符和多行内容，它们是循环的典型场景。假设文件内容能装进内存，你可以用第 9 章文件对象的 `read` 方法一次全部加载：`python file = open('data.txt') # 一次性把内容读入字符串print(file.read())` 要更细粒度地访问，可以用下面任一方式按字符扫描——其中第二种不会加载整个文件，而且因第 11 章的 `:=` 命名

赋值而变得更简练：`python for char in open('data.txt').read(): print(char, end='')` `file = open('data.txt')` `while c`
`har := file.read(1): # 按字符读取，空表示文件结尾``print(c`
`har, end='')` # 每个字符后不要加 `\n` 要改按行或块读取，
可以用下面这样的 `while` 循环；二进制数据常用二进制模
式 `'rb'` 按块读取，但文本应该用文本模式，以免拆裂字符的
字节：`python file = open('data.txt')` `while line := file.r`
`eadline(): # 逐行读取``print(line.rstrip())` # 行已经有 `\n` 换
行符`file = open('data.txt')` `while chunk := file.read(10):`
逐块读取：最多 10 个字符`print(chunk, end='')` # 保留
但不添加换行符不过，要按行读文本文件，`for` 循环往往最
容易编码，也可能运行最快：`python for line in open('da`
`ta.txt').readlines(): print(line.rstrip())` `for line in open('`
`data.txt'):` # 使用迭代器：文本输入的最佳选择（也许）`pr`
`int(line.rstrip())` 这里的第一个版本用文件的 `readlines` 方
法把文件一次性全部加载成行字符串列表，而第二个示例依
赖文件迭代器（`iterator`）在每次循环迭代时自动读取一行
。第二个示例对文本文件来说通常也是最佳选择——除了
简单之外，它还能处理任意大的文件，因为它不会一次性把
整个文件加载进内存。迭代器版本可能也是最快的，尽管速
度会随 Python 版本变化（我们会在本书后面学习给代码计
时的方法）。不过，文件 `readlines` 调用仍然有用——例
如要反转文件的行，前提是内容能装进内存。内置 `reverse`
`d` 只作用于序列，不接受生成值的可迭代对象；`sorted` 则
相反，能接受可迭代对象，因此无需完整加载就能给文件的
所有行排序：`python for line in reversed(open('data.txt`
`').readlines()): print(line.rstrip())` `for line in sorted(ope`
`n('data.txt')):` `print(line.rstrip())` 关于这里用到的文件对象

调用的更多内容，参见 Python 文档——以及它对 `os.popen` 这类工具的介绍：`os.popen`（默认情况下）返回一个连接到 shell 命令输出的文件对象，因此支持同样的循环。第 21 章还有一个 `os.popen` 示例，当我们深入钻研 Unicode 时，第 37 章会更多讨论文本与二进制文件的区别。

代码分析

```
file = open('data.txt')           #
print(file.read())
```

·解释器如何处理：`file.read()` 直接一次性读出全部字节并解码为字符串——简单，但大文件会把内存占满（本书反复提醒）。适合小文件的偷懒读法。

```
file = open('data.txt')
while char := file.read(1):       #
    print(char, end='')           # \n
```

·解释器如何处理：`file.read(1)` 每次读 1 个字符；到文件尾读不到内容时返回空字符串""（假值），`while char := ..` 的海象测试随即结束循环——EOF 用"空返回值"表达，是无病无害的惯用法。注意 `open('data.txt').read()` 是"先全读再逐字符迭代"，而 `read(1)` 版是"读一个打一个"，内存恒定。

```
for line in open('data.txt'):     #
    print(line.rstrip())
```

·解释器如何处理（最关键的一个循环）：文件对象实现迭代协议——`for line in file` 每轮 `next` 就从文件流中读取一行（以换行符为界），直到 EOF 抛 `StopIteration`。因此：① 不整文件占内存；② 惰性逐行 = 天然支持无限大

文件；③ 作者留了一手"（也许）"——速度随版本浮动，第 21 章学计时后才能下结论。

· `readlines` / `reversed` / `sorted` 的区别：`readlines()` 一次性读成行列表（要反转就 `reversed()`，全读入内存）；`sorted(open('f'))` 不需要反转/读取顺序，`sorted` 能接受迭代器并缓存排序——不整读但整排序，仍然要有足够内存装全部行。

深度理解

· 核心概念：文件扫描的三个层次——全读（`read/readlines`）、按行迭代（`for line in file`）、按字符/块（`read(1)/read(n)`）。选型的本质是"内存 vs 简单"的权衡。

· 底层实现：文件迭代器每次从系统缓冲里取一行，内部可能只做少量字符扫描；`read(10)` 则按固定块读。EOF 用 `return`（方法）或 `StopIteration`（迭代器）两种方式表达，导出两种写法。

· 为什么这样设计："文件即迭代器"让第 13 章循环直接吃下"逐行处理日志"这一最常用需求——文件不是一个必须先读干的容器，而是一个能按需吐行的流（`stream`）。

· 实际问题：日志分析（逐行 `grep`/解析）、CSV/配置文件逐行处理、任意大文件的流式处理、管道命令输出捕获（`os.popen`）。

· 初学者误区：① 一律 `file.read()` 全读，内存爆炸；② 用 `readlines` 后还遍历索引而不是直接遍历行；③ 忘记 `rstrip()`——每行自带换行符，直接 `print` 会空两行。

·生活例子：文件迭代器像"自动喂料机"——一行一行送到你面前，喂完自动停；`read()` 像"整袋倒桌上一把抓"——方便但一下子占满整张桌子（内存）。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

·数据处理管道：`for + enumerate/zip` 是数据分析（配序列号、并行对齐）的第一板斧；`for line in open(...)` 是日志、CSV、JSONL 的流式处理标配。

·搜索引擎/匹配：`for...else` 模式天然表达"全表扫描未命中"——如用户权限检查、黑名单过滤、字典键存在性探测。

·数值与算法：`range/len` 定位索引用于原地修改列表（批量更新）、序列旋转（密码学轮换、环形缓冲区显示）、排列生成。

·配置文件与结果集：`dict(zip(keys, vals))` 一行完成"两列合成映射"，是数据库查询列、HTTP 头、表格导出的高频写法。

·交互与状态机：`while True + 底部 break` 实现"先执行后判断"的 `do-until` 语义，是菜单循环、事件驱动读输入的标准骨架。

与其他语言 (Java/C++) 的区别

特性	Python	Java / C++
for 的本质	遍历"可迭代对象", 自动取项	计数器语法 (for (i=0; ...)), 遍历需自己配索
while/for 的 else	有 (独有特性)	无, 需 flag 变量模拟"未中断"
break/continue	跳出/跳到最近一层循环	语义相似, 另有多层 goto/labeled break (Java)
循环变量作用域	循环体共享外层作用域, 结束后变量仍存活	for 头部变量通常块内作用域 (C++; Java 类似)
迭代机制	统一迭代协议 (iter/next), 鸭子类型	集合框架接口 (Iterator), 无内置统一协议的普通数组
range 对象	惰性虚拟序列 (3.X)	C++ 无内置; Java IntStream.range (Java 8+) 类似
遍历索引	用 enumerate/range (len)	通常直接 for (i=0; i< arr.length; i++) 天然有索引

Python 发展历史背景

- 迭代协议的诞生: Python 2.2 (2001 年) 引入统一的迭代协议与生成器, 之前 for 只能遍历序列与 dict 的键列表, 之后任何实现协议的对象 (文件、字典视图、生成器) 都能被 for。
- xrange → range: Python 2 的 range 返回真列表 (大范围占满内存), xrange 惰性; Python 3 把惰性统一进 range, 并消除 xrange——"默认大内存友好"是现代 Python 的思想。
- 海象运算符 :=: PEP 572 (Python 3.8, 2019 年) 引入

赋值表达式，本章用它精简了 while 循环；它的争议（可读性 vs 简洁）正是 13.3.7 节作者自嘲“招来草叉”的由来。

·字典视图 items()：Python 3 把 dict 的键/值/项改为视图对象（惰性、动态），配合 for (k, v) in D.items() 的元组解包成为遍历字典的标准写法。

高级开发者需要掌握的相关知识

·迭代协议的完整细节（下一章主题）：iter/next、StopIteration、可迭代 vs 迭代器、next(it) 内置。

·推导式家族：列表/字典/集合推导与生成器表达式——它们与 for 同构但更快（C 级迭代），是“让 Python 干活”的进阶。

·itertools 标准库：ziplongest（补齐）、islice（惰性切片）、count/cycle/repeat（无限迭代）、chain（合并）——本章技巧的生产级延伸。

·性能度量（第 21 章 preview）：用 timeit/perf 实证“for 比 while 快”“迭代器逐行 vs readlines”等断言，而不是记结论。

·生成器与协程：生成器函数 yield 是所有迭代工具的最终形态（第 20 章），理解它之前必须先懂本章的“惰性与逐项”心智模型。

·异常控制流（第 36 章 preview）：用 try/except 模拟多重跳出嵌套循环——组合 break 的局限。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) ——循环是程序三大控制流之一，本章是全书使用频率最高的语法块（几乎后面每章代码都在用 for 循环），必须零死角掌握。

·应该掌握到什么程度：

1. 能用 while 与 for 熟练互换表达同一逻辑，并默认选择 for；
2. 脱口而出 break（跳出）/continue（跳本轮）/else（未 break 正常退出）三者语义，以及"else 属于循环不属于 if"的缩进规则；
3. 能用 range(len(seq)) 改列表（L[i] += 1），并懂得为什么 for x in L: x += 1 无效；
4. 熟练使用 zip（并行遍历、字典构造）、enumerate（带序号遍历）并理解它们都是惰性迭代器（要用 list() 才「显形」）；
5. 能默写 for line in open(...) 逐行处理文件，并知道全读/逐行/逐块三者的内存差异。

·建议掌握的进阶深度：至少能手写一个"类 enumerate 的迭代器"（下一章标准作业），并从字节码层面解释 for 的实现（iter()+next()+StopIteration）。

·后续应该学习哪些相关内容：

1. 立即接着学：第 14 章"迭代协议与列表推导"——本章所有"先看着用、原理后补"的谜底（range 为何惰性、for 如何吃下文件、推导式为何更快）。

2. 中期：函数（把 seq1/seq2 那段循环泛化成可复用工具）、生成器与 yield（第 20 章）。

3. 进阶：itertools 迭代工具箱、in 运算符在容器上的复杂度差异（list 线性 vs set 哈希）、性能度量的实践（第 21 章）。

4. 相关复习：第 11 章的赋值机制（元组解包/扩展解包/海象）——本章 for 目标的灵活性全靠它。

·练习建议：动手把本章每段 REPL 代码跑一遍并追踪变化；完成书末 Part IV 的质数练习（把 13.3.8 的素数检测包装成函数）；自拟一个问题："不用 range/len，如何打印列表索引？"以逼自己掌握 enumerate。